

OSGi Working Group
**OSGi Specification repo for
`org.osgi.service.remoteser-`**

**Release 1.1.1-SNAPSHOT
September 2026**



LICENSE

Eclipse Foundation Specification License – v1.0

By using and/or copying this document, or the Eclipse Foundation document from which this statement is linked, you (the licensee) agree that you have read, understood, and will comply with the following terms and conditions:

Permission to copy, and distribute the contents of this document, or the Eclipse Foundation document from which this statement is linked, in any medium for any purpose and without fee or royalty is hereby granted, provided that you include the following on ALL copies of the document, or portions thereof, that you use:

- link or URL to the original Eclipse Foundation document.
- All existing copyright notices, or if one does not exist, a notice (hypertext is preferred, but a textual representation is permitted) of the form: "Copyright © [date-of-document] Eclipse Foundation, Inc. <<url to this license>>"

Inclusion of the full text of this NOTICE must be provided. We request that authorship attribution be provided in any software, documents, or other items or products that you create pursuant to the implementation of the contents of this document, or any portion thereof.

No right to create modifications or derivatives of Eclipse Foundation documents is granted pursuant to this license, except anyone may prepare and distribute derivative works and portions of this document in software that implements the specification, in supporting materials accompanying such software, and in documentation of such software, PROVIDED that all such works include the notice below. HOWEVER, the publication of derivative works of this document for use as a technical specification is expressly prohibited.

The notice is:

"Copyright © [date-of-document] Eclipse Foundation. This software or document includes material copied from or derived from [title and URI of the Eclipse Foundation specification document]."

Disclaimers

THIS DOCUMENT IS PROVIDED "AS IS," AND THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION MAKE NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NON-INFRINGEMENT, OR TITLE; THAT THE CONTENTS OF THE DOCUMENT ARE SUITABLE FOR ANY PURPOSE; NOR THAT THE IMPLEMENTATION OF SUCH CONTENTS WILL NOT INFRINGE ANY THIRD PARTY PATENTS, COPYRIGHTS, TRADEMARKS OR OTHER RIGHTS.

THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION WILL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, SPECIAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF ANY USE OF THE DOCUMENT OR THE PERFORMANCE OR IMPLEMENTATION OF THE CONTENTS THEREOF.

The name and trademarks of the copyright holders or the Eclipse Foundation may NOT be used in advertising or publicity pertaining to this document or its contents without specific, written prior permission. Title to copyright in this document will at all times remain with copyright holders.

Preface

Implementation Requirements

An implementation of a Specification: (i) must fully implement the Specification including all its required interfaces and functionality; (ii) must not modify, subset, superset or otherwise extend the OSGi Name Space, or include any public or protected packages, classes, Java interfaces, fields or methods within the OSGi Name Space other than those required and authorized by the Specification. An implementation that does not satisfy limitations (i) (ii) is not considered an implementation of the Specification and must not be described as an implementation of the Specification. "OSGi Name Space" shall mean the public class or interface declarations whose names begin with "org.osgi" or any recognized successors or replacements thereof. An implementation of a Specification must not claim to be a compatible implementation of the Specification unless it passes the Technology Compatibility Kit ("TCK") for the Specification.

Feedback

This specification can be downloaded from the OSGi Documentation web site:

<https://docs.osgi.org/specification/>

Comments about this specification can be raised at:

<https://github.com/osgi/osgi/issues>

Table of Contents

1	Introduction	5
1.1	Reader Level	5
1.2	Version Information	5
1.3	References	5
122	Remote Service Admin Service Specification	7
122.1	Introduction	7
122.2	Actors	10
122.3	Topology Managers	11
122.4	Endpoint Description	12
122.5	Remote Service Admin	16
122.6	Discovery	21
122.7	Events	25
122.8	Endpoint Description Extender Format	27
122.9	Capability Namespaces	33
122.10	Advice to implementations	34
122.11	Security	35
122.12	org.osgi.service.remoteserviceadmin	36
122.13	org.osgi.service.remoteserviceadmin.namespace	53
122.14	References	54

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

1 Introduction

This document contains a specification developed by the OSGi working group.

1.1 Reader Level

This specification is written for the following audiences:

- Application developers
- Framework and system service developers (system developers)
- Architects

This specification assumes that the reader has at least one year of practical experience in writing Java programs. Experience with embedded systems and server-environments is a plus. Application developers must be aware that the OSGi environment is significantly more dynamic than traditional desktop or server environments.

System developers require a very deep understanding of Java. At least three years of Java coding experience in a system environment is recommended. A Framework implementation will use areas of Java that are not normally encountered in traditional applications. Detailed understanding is required of class loaders, garbage collection, Java 2 security, and Java native library loading.

Architects should focus on the introduction of each subject. This introduction contains a general overview of the subject, the requirements that influenced its design, and a short description of its operation as well as the entities that are used. The introductory sections require knowledge of Java concepts like classes and interfaces, but should not require coding experience.

Most of these specifications are equally applicable to application developers and system developers.

1.2 Version Information

Each OSGi specification has an independent version, and packages defined by those specifications have their own versions as well. Specification and package versions follow semantic versioning rules.

1.3 References

- [1] *OSGi Specifications*
<https://docs.osgi.org/specification/>

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

122 Remote Service Admin Service Specification

Version 1.1

122.1 Introduction

The ??? framework specifies a model where bundles can use distributed services. The basic model for OSGi remote services is that a bundle can register services that are *exported* to a communication *Endpoint* and use services that are *imported* from a communication *Endpoint*. However, chapter ??? does not explain *what* services are exported and/or imported; it leaves such decisions to the distribution provider. The distribution provider therefore performs multiple roles and cannot be leveraged by other bundles in scenarios that the distribution provider had not foreseen.

The primary role of the distribution provider is purely mechanical; it creates *Endpoints* and registers service proxies and enables their communication. The second role is about the policies around the desired topology. The third role is discovery. To establish a specific topology it is necessary to find out about exported services in other frameworks.

This specification therefore defines an API for the distribution provider and discovery of services in a network. A management agent can use this API to provide an actual distribution policy. This management agent, called the *Topology Manager*, can control the export and import of services delegating the intrinsic knowledge of the low level details of communication protocols, proxying of services, and discovering services in the network to services defined in this specification.

This specification is an extension of the Remote Service chapter. Though some aspects are repeated in this specification, a full understanding of the Remote Services chapter is required for full understanding of this document.

122.1.1 Essentials

- *Simple* - Make it as simple as possible for a *Topology Manager* to implement distribution policies.
- *Dynamic* - Discover available *Endpoints* dynamically, for example through a discovery protocol like [3] *Service Location Protocol (SLP)* or [4] *JGroups* .
- *Inform* - Provide a mechanism to inform other parties about created and removed *Endpoints*.
- *Configuration* - Allow bundles to describe *Endpoints* as a bundle resource that are provided to the *Distribution Provider*.
- *Selective* - Not all parties are interested in all services. *Endpoint registries* must be able to express the scope of services they are interested in.
- *Multiple* - Allow the collaboration of multiple *Topology Managers*, *Remote Service Admin* services, and *Discovery Providers*.
- *Dynamic* - Allow the dynamic discovery of *Endpoints*.
- *Federated* - Enable a global view of all available services in a distributed environment.

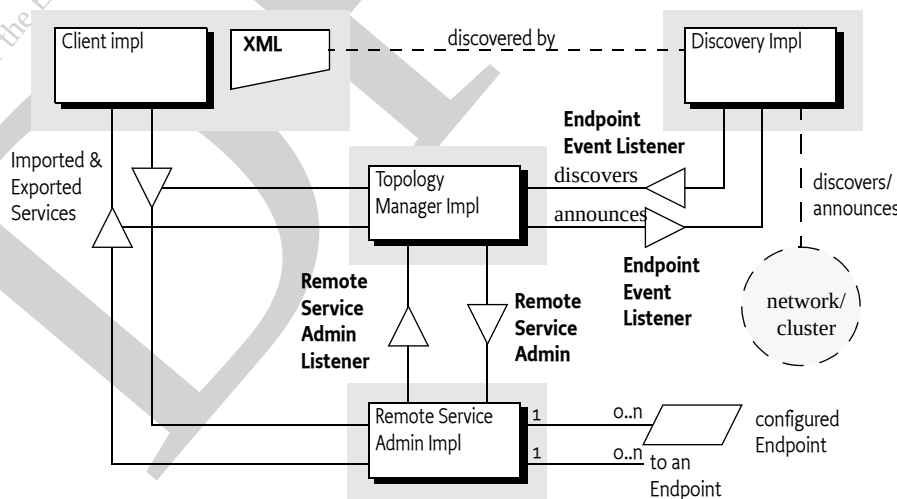
122.1.2

Entities

- *Remote Service Admin* - An implementation of this specification provides the mechanisms to import and export services through a set of configuration types. The Remote Service Admin service is a passive Distribution Provider, not taking any action to export or import itself.
- *Topology Manager* - The Topology Manager provides the policy for importing and exporting services through the Remote Service Admin service.
- *Endpoint* - An Endpoint is a communications access mechanism to a service in another framework, a (web) service, another process, or a queue or topic destination, etc., requiring some protocol for communications.
- *Endpoint Description* - A properties based description of an Endpoint. Endpoint Descriptions can be exchanged between different frameworks to create connections to each other's services. Endpoint Descriptions can also be created to Endpoints not originating in an OSGi Framework.
- *Endpoint Description Provider* - A party that can inform others about the existence of Endpoints.
- *Endpoint Event Listener* - A listener service that receives events relating to Endpoints that match its scope. This Endpoint Event Listener is used symmetrically to implement a federated registry. The Topology Manager can use it to notify interested parties about created and removed Endpoints, as well as to receive notifications from other parties, potentially remote, about their available Endpoints.
- *Endpoint Listener* - An older version of the Endpoint Event Listener defined by version 1.0 of this specification. The Endpoint Event Listener supersedes the Endpoint Listener, and should be used in preference where possible.
- *Remote Service Admin Listener* - A listener service that is informed of all the primitive actions that the Remote Service Admin performs like importing and exporting as well as errors.
- *Endpoint Configuration Extender* - A bundle that can detect configuration data describing an Endpoint Description in a bundle resource, using the extender pattern.
- *Discovery* - An Endpoint Event Listener that detects the Endpoint Descriptions through some discovery protocol.
- *Cluster* - A group of computing systems that closely work together, usually in a fast network.

Figure 122.1

Remote Service Admin Entities



122.1.3

Synopsis

Topology Managers are responsible for the distribution policies of a OSGi framework. To implement a policy, a Topology Manager must be aware of the environment, for this reason, it can register:

- Service listeners to detect services that can be exported according to the Remote Services chapter.
- Listener and Find Hook services to detect bundles that have an interest in specific services that potentially could be imported.
- A Remote Service Admin Listener service to detect the activity of other Topology Managers.
- Endpoint Event Listener and Endpoint Listener services to detect Endpoints that are made available through discovery protocols, configuration data, or other means.

Using this information, the manager implements a topology using the Remote Service Admin service. A Topology Manager that wants to export a service can create an *Export Registration* by providing one or more Remote Service Admin services a Service Reference plus a Map with the required properties. A Remote Service Admin service then creates a number of Endpoints based on the available configuration types and returns a collection of ExportRegistration objects. A collection is returned because a single service can be exported to multiple Endpoints depending on the available configuration type properties.

Each Export Registration is specific for the caller and represents an existing or newly created Endpoint. The Export Registration associates the exported Service Reference with an *Endpoint Description*. If there are problems with the export operation, the Remote Service Admin service reports these on the Export Registration objects. That is, not all the returned Export Registrations have to be valid.

An Endpoint Description is a property based description of an Endpoint. Some of these properties are defined in this specification, other properties are defined by configuration types. These configuration types must follow the same rules as the configuration types defined in the Remote Services chapter. Remote Service Admin services that support the configuration types in the Endpoint Description can import a service from that Endpoint solely based on that Endpoint Description.

In similar vein, the Topology Manager can import a service from a remote system by creating an Import Registration out of an Endpoint Description. The Remote Service Admin service then registers a service that is a proxy for the remote Endpoint and returns an ImportRegistration object. If there are problems with the import, the Remote Service Admin service that cannot be detected early, then the Remote Service Admin service reports these on the returned ImportRegistration object.

For introspection, the Remote Service Admin can list its current set of Import and Export References so that a Topology Manager can get the current state. The Remote Service Admin service also informs all Topology Managers and observers of the creation, deletion, and errors of Import and Export Registrations through the Remote Service Admin Listener service. Interested parties like the Topology Manager can register such a service and will be called back with the initial state as well as any subsequent changes.

An important aspect of the Topology Manager is the distributed nature of the scenarios it plays an orchestrating role in. A Topology Manager needs to be aware of Endpoints in the network, not just the ones provided by Remote Service Admin services in its local framework. The Endpoint Event Listener service is specified for this purpose. This service is provided for both directions, symmetrically. That is, it is used by the Topology Manager to inform any observers about the existence of Endpoints that are locally available, as well as for parties that represent a discovery mechanism. For example Endpoints available on other systems, Endpoint Descriptions embedded in resources in bundles, or Endpoint Descriptions that are available in some other form.

Endpoint Event Listener services are not always interested in the complete set of available Endpoints because this set can potentially be very large. For example, if a remote registry like [5] UDDI is used then the number of Endpoints can run into the thousands or more. An Endpoint Event Listener service can therefore scope the set of Endpoints with an OSGi LDAP style filter. Parties that can provide information about Endpoints must only notify Endpoint Event Listener services when the Endpoint Description falls within the scope of the Endpoint Listener service. Parties that use some discovery mechanism can use the scope to trigger directed searches across the network.

122.1.3.1

Endpoint Listener Services

The 1.0 version of this specification defined an Endpoint Listener service, which has an identical purpose and similar behaviors to an Endpoint Event Listener service. Unfortunately the design of the Endpoint Listener limited its extensibility, meaning that it had to be replaced in version 1.1 of this specification.

In order to maintain backward compatible interoperability with Remote Service Admin 1.0 actors, Remote Service Admin 1.1 actors must continue to register Endpoint Listener services as well as Endpoint Event Listener services. They must also continue to call Endpoint Listener services as well as EndpointEventListener services.

122.2

Actors

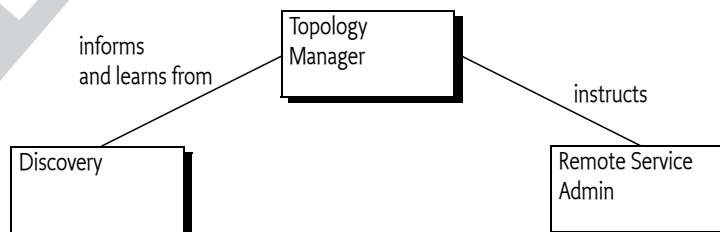
The OSGi Remote Services specification is about the distribution of services. This specification does not outline the details of how the distribution provider knows the desired topology, this policy aspect is left up to implementations. In many situations, this is a desirable architecture because it provides freedom of implementation to the distribution provider. However, such an architecture does not enable a separation of the mechanisms and *policy*. Therefore, this Remote Service Admin specification provides an architecture that enables a separate bundle from the distribution provider to define the topology. It splits the responsibility of the Remote Service specification in a number of *roles*. These roles can all have different implementations but they can collaborate through the services defined in this specification. These roles are:

- *Topology Managers* - Topology Managers are the (anonymous) players that implement the policies for distributing services; they are closely aligned with the concept of an OSGi *management agent*. It is expected that Topology Managers will be developed for scenarios like import/export all applicable services, configuration based imports- and exports, and scenarios like fail-over, load-balancing, as well as standards like domain managers for the [6] *Service Component Architecture (SCA)*.
- *Remote Service Admin* - The Remote Service Admin service provides the basic mechanism to import and export services. This service is policy free; it will not distribute services without explicitly being told so. A OSGi framework can host multiple Remote Service Admin services that, for example, support different configuration types.
- *Discovery* - To implement a distribution policy, a Topology Manager must be aware of what Endpoints are available. This specification provides an abstraction of a *federated Endpoint registry*. This registry can be used to both publish as well as consume Endpoints from many different sources. The federated registry is defined for local services but is intended to be used with standard and proprietary service discovery protocols. The federated registry is implemented with the Endpoint Event Listener service.

These roles are depicted in Figure 122.2 on page 10 .

Figure 122.2

Roles



122.3 Topology Managers

Distributed processing has become mainstream because of the massive scale required for Internet applications. Only with distributed architectures is it possible to scale systems to *Internet size* with hundreds of millions of users. To allow a system to scale, servers are grouped in clusters where they can work in unison or geographically dispersed in even larger configurations. The distribution of the work-load is crucial for the amount of scalability provided by an architecture and often has domain specific dispatching techniques. For example, the hash of a user id can be used to select the correct profile database server. In this fast moving world it is very unlikely that a single architecture or distribution policy would be sufficient to satisfy many users. It is therefore that this specification separates the *how* from the *what*. The complex mechanics of importing and exporting services are managed by a Remote Service Admin service (the *how*) while the different policies are implemented by Topology Managers (the *what*). This separation of concerns enables the development of Topology Managers that can run on many different systems, providing high user functionality. For example, a Topology Manager could implement a fail-over policy where some strategic services are redirected when their connections fail. Other Topology Managers could use a discovery protocol like SLP to find out about other systems in a cluster and automatically configure the cluster.

The key value of this architecture is demonstrated by the example of an *SCA domain controller*. An SCA domain controller receives a description of a domain (a set of systems and modules) and must ensure that the proper connections are made between the participating SCA modules. By splitting the roles, an SCA domain manager can be developed that can run on any compatible Remote Service Admin service implementation.

122.3.1 Multiple Topology Managers

There is no restriction on the number of Topology Managers, nor is there a restriction on the number of Remote Service Admin service implementations. It is up to the deployer of the OSGi framework to select the appropriate set of these service implementations. It is the responsibility of the Topology Managers to listen to the Remote Service Admin Listener and track Endpoints created and deleted by other Topology Managers and act appropriately.

122.3.2 Example Use Cases

122.3.2.1 Promiscuous Policy

A *cluster* is a set of machines that are connected in a network. The simplest policy for a Topology Manager is to share exported services in such a cluster. Such a policy is very easy to implement with the Remote Services Admin service. In the most basic form, this Topology Manager would use some multicast protocol to communicate with its peers. These peers would exchange EndpointDescription objects of exported services. Each Topology Manager would then import any exported service.

This scenario can be improved by separating the promiscuous policy from the discovery. Instead of embedding the multicast protocol, a Topology manager could use the Endpoint Event Listener service. This service allows the discovery of remote services. At the same time, the Topology Manager could tell all other Endpoint Event Listener services about the services it has created, allowing them to be used by others in the network.

Splitting the Topology Manager and discovery in two bundles allows different implementations of the discovery bundle, for example, to use different protocols. See [PROMISCUOUS_POLICY](#).

122.3.2.2 Fail Over

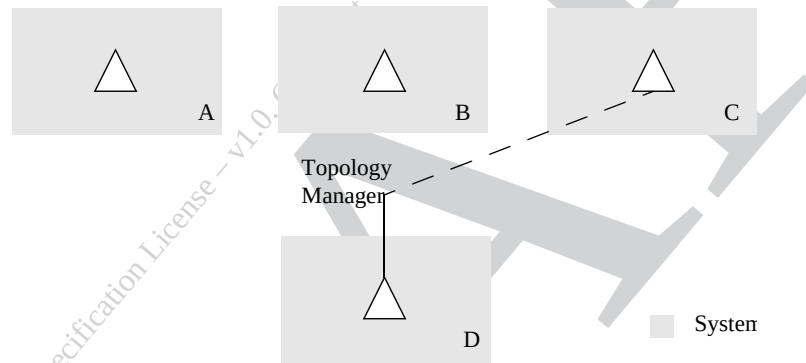
A more elaborate scheme is a *fail-over policy*. In such a policy a service can be replaced by a service from another machine. There are many ways to implement such a policy, an simple example strategy is provided here for illustration.

A Fail-Over Topology Manager is given a list of stateless services that require fail-over, for example through the `FailOver` interface. The Fail-Over Manager tracks the systems in its cluster that provide such services. This tracking can use an embedded protocol or it can be based on the Endpoint Event Listener service model.

In the Fail-Over policy, the fail-over manager only imports a single service and then tracks the error status of the imported service through the Remote Service Admin Listener service. If it detects the service is becoming unavailable, it closes the corresponding Import Registration and imports a service from an alternative system instead. In Figure 122.3, there are 4 systems in a cluster. The topology/fail-over manager ensures that there is always one of the services in system A, B, or C available in D.

Figure 122.3

Fail Over Scenario in a cluster



There are many possible variations on this scenario. The managers could exchange load information, allowing the service switch to be influenced by the load of the target systems. The important aspect is that the Topology Manager can ignore the complex details of discovery protocols, communication protocols, and service proxying and instead focus on the topology. See [FAIL_OVER_POLICY](#).

122.4 Endpoint Description

An *Endpoint* is a point of rendezvous of distribution providers. It is created by an exporting distribution provider or some other party, and is used by importing distribution providers to create a connection. An *Endpoint Description* describes an Endpoint in such a way that an importing Remote Service Admin service can create this connection if it recognizes the *configuration type* that is used for that Endpoint. The configuration type consists of a name and a set of properties associated with that name.

The core concept of the Endpoint Description is a Map of properties. The structure of this map is the same as service properties, and the defined properties are closely aligned with the properties of an imported service. An EndpointDescription object must only consist of the data types that are supported for service properties. This makes the property map serializable with many different mechanisms. The EndpointDescription class provides a convenient way to access the properties in a type safe way.

An Endpoint Description has case insensitive keys, just like the Service Reference's properties.

The properties map must contain all the prescribed service properties of the exported service after intents have been processed, as if the service was registered as an imported service. That is, the map must not contain any properties that start with `service.exported.*` but it must contain the `service.imported.*` variation of these properties. The Endpoint Description must reflect the imported service properties because this simplifies the use of filters from the service hooks. Filters applied to

the Endpoint Description can then be the same filters as applied by a bundle to select an imported service from the service registry.

The properties that can be used in an Endpoint Description are listed in Table 122.1. The RemoteConstants class contains the constants for all of these property names.

Table 122.1 *Endpoint Properties*

Endpoint Property Name	Type	Description
service.exported.*		Must not be set
service.imported	*	Must always be set to some value. See SERVICE_IMPORTED .
objectClass	String[]	Must be set to the value of service.exported.interfaces, of the exported service after expanding any wildcards. Though this property will be overridden by the framework for the corresponding service registration, it must be set in the Endpoint Description to simplify the filter matching. These interface names are available with the getInterfaces() method.
service.intents	String+	Intents implemented by the exporting distribution provider and, if applicable, the exported service itself. Any qualified intents must have their expanded form present. These expanded intents are available with the getIntents() method. See SERVICE_INTENTS .
endpoint.service.id	Long	The service id of the exported service. Can be absent or 0 if the corresponding Endpoint is not for an OSGi service. The remote service id is available as getServiceId() . See also ENDPOINT_SERVICE_ID .
endpoint.framework.uuid	String	A universally unique id identifying the instance of the exporting framework. Can be absent if the corresponding Endpoint is not for an OSGi service. See <i>Framework UUID</i> on page 15. The remote framework UUID is available with the getFrameworkUUID() method. See also ENDPOINT_FRAMEWORK_UUID .
endpoint.id	String	The Id for this Endpoint, can never be null. This information is available with the getId() . See <i>Endpoint Id</i> on page 15 and also ENDPOINT_ID .
endpoint.package.version.<package-name>	String	The Java package version for the embedded <package>. For example, the property endpoint.package.version.com.acme=1.3 describes the version for the com.acme package. The version for a package can be obtained with the getPackageVersion(String) . The version does not have to be set, if not set, the value must be assumed to be 0.
service.imported.configs	String+	The configuration types that can be used to implement the corresponding Endpoint. This property maps to the corresponding property in the Remote Services chapter. This property can be obtained with the getConfigurationTypes() method. The Export Registration has all the possible configuration types, where the Import Registration reports the configuration type actually used. SERVICE_IMPORT_ED_CONFIGS .

Endpoint Property Name	Type	Description
<config>.*	*	Where <config> is one of the configuration type names listed in <code>service.imported.configs</code> . The content of these properties must be valid for creating a connection to the Endpoint in another framework. That is, any locally readable URLs from bundles must be converted in such a form that they can be read by the importing framework. How this is done is configuration type specific.
*	*	All remaining public service properties must be present (that is, not starting with full stop (' \u002E)). If the values can not be marshaled by the Distribution Provider then they must be ignored.

The `EndpointDescription` class has a number of constructors that make it convenient to instantiate it for different purposes:

- [EndpointDescription\(Map\)](#) - Instantiate the Endpoint Description from a Map object.
- [EndpointDescription\(ServiceReference,Map\)](#) - Instantiate an Endpoint Description based on a Service Reference and a Map. The base properties of this Endpoint Description are the Service Reference properties but the properties in the given Map must override any of their case variants in the Service Reference. This allows the construction of an Endpoint Description from an exportable service while still allowing overrides of specific properties by the Topology Manager.

The Endpoint Description must use the allowed properties as given in Table 122.1 on page 13 . The Endpoint Description must automatically skip any `service.exported.*` properties.

The Endpoint Description provides the following methods to access the properties in a more convenient way:

- [getInterfaces\(\)](#) - Answers a list of Java interface names. These are the interfaces under which the services must be registered. These interface names can also be found at the `objectClass` property. A service can only be imported when there is at least one Java interface name available.
- [getConfigurationTypes\(\)](#) - Answer the configuration types that are used for exporting this Endpoint. The configuration types are associated with a number of properties.
- [getId\(\)](#) - Returns an Id uniquely identifying an Endpoint. The syntax of this Id should be defined in the specification for the associated configuration type. Two Endpoint Descriptions with the same Id describe the same Endpoint.
- [getFrameworkUUID\(\)](#) - Get a Universally Unique Identifier (UUID) for the framework instance that has created the Endpoint, *Framework UUID* on page 15 .
- [getServiceId\(\)](#) - Get the service id for the framework instance that has created the Endpoint. If there is no service on the remote side the value must be 0.
- [getPackageVersion\(String\)](#) - Get the version for the given package.
- [getIntents\(\)](#) - Get the list of specified intents.
- [getProperties\(\)](#) - Get all the properties.

Two Endpoint Descriptions are deemed equal when their Endpoint Id is equal. The Endpoint Id is a mandatory property of an Endpoint Description, it is further described at *Endpoint Id* on page 15 . The hash code is therefore also based on the Endpoint Id.

122.4.1

Validity

A valid Endpoint Description must at least satisfy the following assertions:

- It must have a non-null Id that uniquely identifies the Endpoint

- It must at least have one Java interface name
- It must at least have one configuration type set
- Any version for the packages must have a valid version syntax.

122.4.2 Mutability

An EndpointDescription object is immutable and with all final fields. It can be freely used between different threads.

122.4.3 Endpoint Id

An Endpoint Id is an opaque unique identifier for an Endpoint. This uniqueness must at least hold for the entire network in which the Endpoint is used. There is no syntax defined for this string except that white space at the beginning and ending must be ignored. The actual syntax for this Endpoint Id must be defined by the actual configuration type.

Two Endpoint Descriptions are deemed identical when their Endpoint Id is equal. The Endpoint Ids must be compared as string compares with leading and trailing spaces removed. The Endpoint Description class must use the String class' hash Code from the Endpoint Id as its own hashCode.

The simplest way to ensure that a growth in the number of EndpointDescriptions and/or the size of the connected group does not violate the required uniqueness of Endpoint Ids is for implementations to make their Endpoint Ids globally unique. This protects against clashes regardless of changes to the connected group.

Whilst globally unique identifiers (GUIDs) are a simple solution to the Endpoint Id uniqueness problem, they are not easy to implement in all environments. In some systems they can be prohibitively expensive to create, or of insufficient entropy to be genuinely unique. Some distribution providers may therefore choose not to use random GUIDs.

In the case where no globally unique value is used the following actions are recommended (although not required).

- Distribution Providers protect against intra-framework clashes using some known value unique to the service, for example the service id.
- Distribution Providers protect against inter-provider collisions within a single framework by using some unique value, such as the distribution provider's bundle id. The distribution provider bundle's symbolic name is insufficient, as there may be multiple versions of the same distribution provider installed within a single framework.
- Distribution Providers protect against inter-framework collisions using some value unique to the framework, such as the framework UUID.

122.4.4 Framework UUID

Each framework registers its services with a service id that is only unique for that specific framework. The OSGi framework is not a singleton, making it possible that a single VM process holds multiple OSGi frameworks. Therefore, to identify an OSGi service uniquely it is necessary to identify the framework that has registered it. This identifier is a *Universally Unique Identifier* (UUID) that is set for each framework. This UUID is contained in the following framework property:

```
org.osgi.framework.uuid
```

If an Endpoint Description has no associated OSGi service then the UUID of that Endpoint Description must not be set and its service id must be 0.

A local Endpoint Description will have its framework UUID set to the local framework. This makes it straightforward to filter for Endpoint Descriptions that are describing local Endpoints or that describe remote Endpoints. For example, a manager can take the filter from a listener and ensure that it is only getting remote Endpoint Descriptions:

```
(
  (!
    (service.remote.framework.uuid
      =72dc5fd9-5f8f-4f8f-9821-9ebb433a5b72)
    )
  (objectClass=org.osgi.service.log.LogService)
)
```

Where 72dc5fd9-5f8f-4f8f-9821-9ebb433a5b72 is the UUID of the local framework. A discovery bundle can register the following filter in its scope to receive all locally generated Endpoints:

```
(service.remote.framework.uuid
  =72dc5fd9-5f8f-4f8f-9821-9ebb433a5b72)
```

122.4.5

Resource Containment

Configuration types can use URLs to point to local resources describing in detail the Endpoint parameters for specific protocols. However, the purpose of an Endpoint Description is to describe an Endpoint to a remote system. This implies that there is some marshaling process that will transfer the Endpoint Description to another process. This other process is unlikely to be able to access resource URLs. Local bundle resource URLs are only usable in the framework that originates them but even HTTP based URLs can easily run into problems due to firewalls or lack of routing.

Therefore, the properties for a configuration type should be stored in such a way that the receiving process can access them. One way to achieve this is to contain the configuration properties completely in the Endpoint Description and ensure they only use the basic data types that the remote services chapter in the core requires every Distribution Provider to support.

The Endpoint Description XML format provides an `xml` element that is specifically added to make it easy to embed XML based configuration documents. The XML Schema is defined in *Endpoint Description Extender Format* on page 27 .

122.5

Remote Service Admin

The Remote Service Admin service abstracts the core functionality of a distribution provider: exporting a service to an Endpoint and importing services from an Endpoint. However, in contrast with the distribution provider of the Remote Services specification, the Remote Service Admin service must be told explicitly what services to import and export.

122.5.1

Exporting

An exportable service can be exported with the `exportService(ServiceReference,Map)` method. This method creates a number of Endpoints by inspecting the merged properties from the Service Reference and the given Map. Any property in the Map overrides the Service Reference properties, regardless of case. That is, if the map contains a key then it will override any case variant of this key in the Service Reference. However, if the Map contains the `objectClass` or `service.id` property key in any case variant, then these properties must not override the Service Reference's value.

The Remote Service Admin service must interpret the merged properties according to the Remote Services chapter. This means that it must look at the following properties (as defined in chapter ???):

- `service.exported.configs` - (String+) A list of configuration types that should be used to export this service. Each configuration type represents the configuration parameters for an Endpoint. A Remote Service Admin service should create an Endpoint for each configuration type that it supports and ignore the types it does not recognize. If this property is not set, then the Remote Service Admin implementation must choose a convenient configuration type that then must be

reported on the Endpoint Description with the service.imported.configs associated with the returned Export Registration.

- service.exported.intents - (String+) A list of intents that the Remote Service Admin service must implement to distribute the given service.
- service.exported.intents.extra - (String+) This property is merged with the service.exported.intents property.
- service.exported.interfaces - (String+) This property must be set; it marks this service for export and defines the interfaces. The list members must all be contained in the types listed in the objectClass service property from the Service Reference. The single value of an asterisk ('*\u002A') indicates all interfaces in the registration's objectClass property and ignore the classes. Being able to set this property outside the Service Reference implies that the Topology Manager can export any registered service, also services not specifically marked to be exported.
- service.intents - (String+) A list of intents that this service has implemented.

A Topology Manager cannot remove properties, null is invalid as a property value.

The Remote Service Admin returns a collection of ExportRegistration objects. This collection must contain an entry for each configuration type the Remote Service Admin has recognized. Unrecognized configuration types must be ignored. Recognized configuration types which require intents that are not supported by the Remote Service Admin must also be ignored. However, it is possible that this list contains *invalid registrations*, see *Invalid Registrations* on page 20 .

If a Service was already exported then the Remote Service Admin must still return a new ExportRegistration object that is linked with the earlier registrations. That is, an Endpoint can be shared between multiple Export Registrations. The Remote Service Admin service must ensure that the corresponding Endpoint remains available as long as there is at least one open Export Registration for that Endpoint.

For each successful creation of an export registration, the Remote Service Admin service must publish an [EXPORT_REGISTRATION](#) event, see *Events* on page 25 . This event must be emitted, even if the Endpoint already existed and is thus shared with another Export Registration. If the creation of an Endpoint runs into an error, an [EXPORT_ERROR](#) event must be emitted.

Each valid Export Registration corresponds to an Endpoint for the given service. This Endpoint must remain active until all of the Export Registrations are closed that share this Endpoint.

The Endpoint can now be published so that other processes or systems can import this Endpoint. To aid with this import, the Export Registration has a [getExportReference\(\)](#) method that returns an [ExportReference](#) object. This reference provides the following information:

- [getExportedEndpoint\(\)](#) - This is the associated Endpoint Description. This Endpoint Description is a properties based description of an Endpoint. The property keys and their semantics are outlined in *Endpoint Description* on page 12 . It can be used to inform other systems of the availability of an Endpoint.
- [getExportedService\(\)](#) - The Service Reference to the exported service.

Both methods must return null when the associated Export Registration is closed.

A Distribution Provider that recognizes the configuration type in an Endpoint can create a connection to an Endpoint on other systems as long as firewalls and networks permit. The Endpoint Description can therefore be communicated to other systems to announce the availability of an Endpoint. The Topology Manager can optionally announce the availability of an Endpoint to the Endpoint Event Listener services, see *Discovery* on page 21 . The decision to announce the availability of an Endpoint is one of the policies that is provided by a specific Topology Manager.

The Export Registrations remain open until:

- Explicitly closed by the Topology Manager, or

- The Remote Service Admin service is no longer used by the Topology Manager that created the Export Registration.

If the Remote Service Admin service can no longer maintain the corresponding Endpoint due to failures than these should be reported through the events. However, the registrations should remain open until explicitly closed by the Topology Manager.

See *Registration Life Cycle* on page 20 for more information.

The Export Registrations are not permanent; persistence is in the realm of the Topology Manager.

122.5.2

Importing

To import a service, a Topology Manager must have an Endpoint Description that describes the Endpoint the imported service should connect to. With this Endpoint Description, a Remote Service Admin service can then import the corresponding Endpoint. A Topology Manager can obtain these Endpoint Descriptions through internal configuration; it can use the discovery model enabled by the Endpoint Event Listener service, see *Discovery* on page 21, or some alternate means.

A service can be imported with the Remote Service Admin `importService(EndpointDescription)` method. This method takes an Endpoint Description and picks one of the embedded configuration types to establish a connection with the corresponding Endpoint to create a local service proxy. This proxy can then be mapped to either a remote OSGi service or an alternative, for example a web service. In certain cases the service proxy can be lazy, only verifying the reachability of the Endpoint when it is actually invoked for the first time. This implies that a service proxy can block when invoked until the proper communication setup has taken place.

If the Remote Service Admin service does not recognize any of the configuration types then it must return null. If there are multiple configuration types recognized then the Remote Service Admin is free to select any one of the recognized types.

The Remote Service Admin service must ensure that service properties are according to the Remote Services chapter for an imported service. This means that it must register the following properties:

- `service.imported` - (*) Must be set to any value.
- `service.imported.configs` - (String+) The configuration information used to import this service. Any associated properties for this configuration types must be properly mapped to the importing system. For example, a URL in these properties must point to a valid resource when used in the importing framework, see *Resource Containment* on page 16. Multiple configuration types can be listed if they are synonyms for exactly the same Endpoint that is used to export this service.
- `service.intents` - (String+) The Remote Service Admin must set this property to convey the combined intents of:
 - The exporting service, and
 - The intents that the exporting distribution provider adds, and
 - The intents that the importing distribution provider adds.
- Any additional properties listed in the Endpoint Description that should not be excluded. See *Endpoint Description* on page 12 for more details about the properties in the Endpoint Description.

A Remote Service Admin service must strictly follow the rules for importing a service as outlined in the Remote Services chapter.

The Remote Service Admin must return an `ImportRegistration` object or null. Even if an `ImportRegistration` is returned, it can still be an *invalid registration*, see *Invalid Registrations* on page 20 if the setup of the connection failed asynchronously. The Import Registration must always be a new object. Each valid Import Registration corresponds to a proxy service, potentially shared, that

was created for the given Endpoint. The issues around proxying are described in *Proxying* on page 20 .

For each successful creation of an import registration, the Remote Service Admin service must publish an `IMPORT_REGISTRATION` event, if there is an error it must publish an `IMPORT_ERROR`, see *Events* on page 25 .

For more information see *Registration Life Cycle* on page 20 .

The Import Registration provides access to an `ImportReference` object with the `getImportReference()` . This object has the following methods:

- `getImportedEndpoint()` - Provides the Endpoint Description for this imported service.
- `getImportedService()` - Provides the Service Reference for the service proxy.

The Import Registration will remain open as long as:

- The corresponding remote Endpoint remains available, and
- The Remote Service Admin service is still in use by the Topology Manager that created the Import Registration.

That is, the Import Registrations are not permanent, any persistence is in the realm of the Topology Manager. See *Registration Life Cycle* on page 20 for more details.

122.5.3

Updates

Services Registrations are dynamic and service properties may change during the lifetime of a service. Remote services must mirror these dynamics without making it appear as though the service has become unavailable. This requires that the exporting distribution provider and the importing distribution provider support the changing of service properties.

There are two types of service properties:

- Properties that are intended to be consumed by the distribution provider, such as: the exported interfaces and configuration types, exported intents and configuration type specific properties. These properties are typically prefixed with 'service.' or 'endpoint.' see Table 122.1 on page 13 .
- Service properties not intended for the distribution provider. These are typically used to communicate information to the consumer of the service and are often specific to the domain of the service.

The following methods to support the updating of service properties on Export Registrations and the propagation of these updates to the remote proxies via Import Registrations.

- `ExportRegistration.update(Map)` - Allows the Topology Manager to update an existing export registration it created after receiving a notification of changed properties on the remotized service.
- `ImportRegistration.update(EndpointDescription)` - Allows the Topology Manager to update the import registration representing a remote service after the remote service properties have been updated. Typically the topology manager is notified of such change via the Discovery mechanism.

The distribution provider must support the updates of service properties *not* intended for the distribution provider, where supported property values are as defined in the *Filter Syntax* of ??? . Distribution providers may support updates to a wider set of properties or data types, but these may fail with other implementations.

122.5.4

Reflection

The Remote Service Admin service provides the following methods to get the list of the current exported and imported services:

- [getExportedServices\(\)](#) - List the Export References for services that are exported by this Remote Service Admin service as directed by any of the Topology Managers.
- [getImportedEndpoints\(\)](#) - List the Import References for services that have been imported by this Remote Service Admin service as directed by any of the Topology Managers.

122.5.5 Registration Life Cycle

All registrations obtained through a Remote Service Admin service are life cycle bound to the Topology Manager that created it. That is, if a Topology Manager ungets its Remote Service Admin service, all registrations obtained through this service must automatically be closed. This model ensures that all registrations are properly closed if either the Remote Service Admin or the Topology Manager stops because in both cases the framework performs the unget automatically. Such behavior can be achieved by implementing the Remote Service Admin service as a Service Factory.

122.5.6 Invalid Registrations

The Remote Service Admin service is explicitly allowed to return *invalid* Import and Export Registrations. First, in a communications stack it can take time to discover that there are issues, allowing the registration to return before it has completed can potentially save time. Second, it allows the Topology Manager to discover problems with the configuration information. Without the invalid Export Registrations, the Topology Manager would have to scan the log or associate the Remote Service Admin Events with a specific import/export method call, something that can be difficult to do.

If the registration is invalid, the [getException\(\)](#) method must return a Throwable object. If the registration has initialized correctly, this method will return null. The [getExportReference\(\)](#) and [getImportReference\(\)](#) methods must throw an Illegal State Exception when the registration is invalid. A Remote Service Admin service is allowed to block for a reasonable amount of time when any of these methods is called, including the [getException](#) method, to finish initialization.

An invalid registration can be considered as never having been opened, it is therefore not necessary to close it; however, closing an invalid or closed registration must be a dummy operation and never throw an Exception. However, a failed registration must generate a corresponding error event.

122.5.7 Proxying

It is the responsibility of the Remote Service Admin service to properly proxy an imported service. This specification does not mandate the technique used to proxy an Endpoint as a service in the OSGi framework. The OSGi Remote Services specification allows a distribution provider to limit what it can proxy.

One of the primary aspects of a proxy is to ensure class space consistency between the exporting bundle and importing bundles. This can require the generation of a proxy-per-bundle to match the proper class spaces. It is the responsibility of the Remote Service Admin to ensure that no Class Cast Exceptions occur.

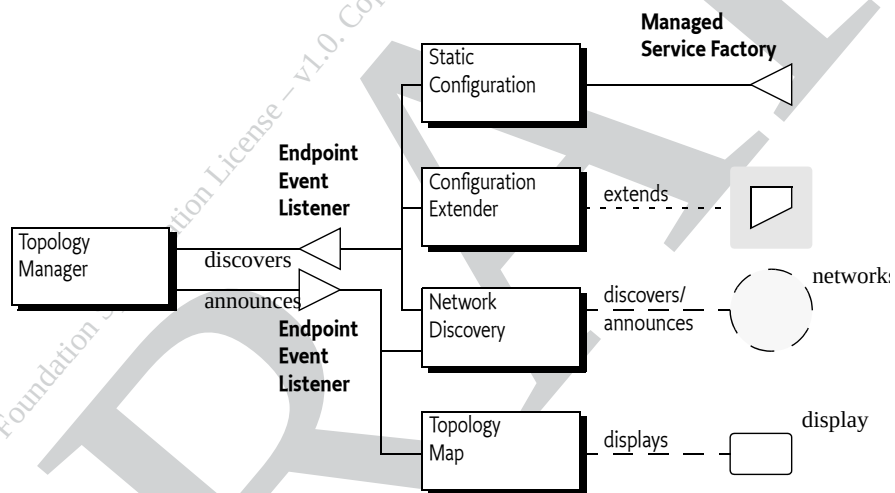
A common technique to achieve maximum class space compatibility is to use a Service Factory. A Service Factory provides the calling bundle when it first gets the service, making it straightforward to verify the package version of the interface that the calling bundle uses. Knowing the bundle that requests the service allows the creation of specialized proxies for each bundle. The interface class(es) for the proxy can then be loaded directly from the bundle, ensuring class compatibility. Interfaces should be loadable by the bundle otherwise that bundle can not use the interface in its code. If an interface cannot be loaded then it can be skipped. A dedicated class loader can then be created that has visibility to all these interfaces and is used to define the proxy class. This design ensures proper visibility and consistency. Implementations can optimize this model by sharing compatible class loaders between bundles.

The proxy will have to call arbitrary methods on arbitrary services. This has a large number of security implications, see *Security* on page 35 .

122.6 Discovery

The topology of the distributed system is decided by the Topology Manager. However, in a distributed environment, the Topology Manager needs to *discover* Endpoints in other frameworks. There is a very large number of ways how a Topology Manager could learn about other Endpoints, ranging from static configuration, a centralized administration, all the way to fully dynamic discovery protocols like the Service Location Protocol (SLP) or JGroups. To support the required flexibility, this specification defines an *Endpoint Event Listener* service that allows the dissemination of Endpoint information. This service provides a symmetric solution because the problem is symmetric: it is used by a Topology Manager to announce changes in its local topology as well as find out about other Endpoint Descriptions. Where those other Endpoint Descriptions come from can vary widely. This design is depicted in Figure 122.4 on page 21 .

Figure 122.4 Examples



The design of the Endpoint Event Listener allows a federated registry of Endpoint Descriptions. Any party that is interested in Endpoint Descriptions should register an Endpoint Event Listener service. This will signal that it is interested in topology information to any *Endpoint Description Providers*. Each Endpoint Event Listener service must be registered with a service property that holds a set of filter strings to indicate the *scope* of its interest. These filters must match an Endpoint Description before the corresponding Endpoint Event Listener service is notified of the availability of an Endpoint Description. Scoping is intended to limit the delivery of unnecessary Endpoint Descriptions as well as signal the need for specific Endpoints.

In addition to providing an Endpoint Event Listener actors must provide an Endpoint Listener. This may, or may not, be the same service object as the Endpoint Event Listener. Registering an Endpoint Listener in addition to an Endpoint Event Listener ensures that Endpoint announcements from version 1.0 actors will continue to be visible. If a service object is advertised as both an Endpoint Listener and an Endpoint Event Listener then version 1.1 actors must use the Endpoint Event Listener interface of the service in preference, and not call it as an Endpoint Listener. For this reason the Endpoint Listener interface is marked as Deprecated. The reason that the Endpoint Event Listener interface should be preferred is that it supports more advanced notification types, such as modification events.

A Topology Manager has knowledge of its local Endpoints and is likely to be only interested in remote Endpoints. It can therefore set the scope to only match remote Endpoint Descriptions. See *Framework UUID* on page 15 for how to limit the scope to local or remote Endpoints. At the

same time, a Topology manager should inform any locally registered Endpoint Event Listener and Endpoint Listener services about Endpoints that it has created or deleted.

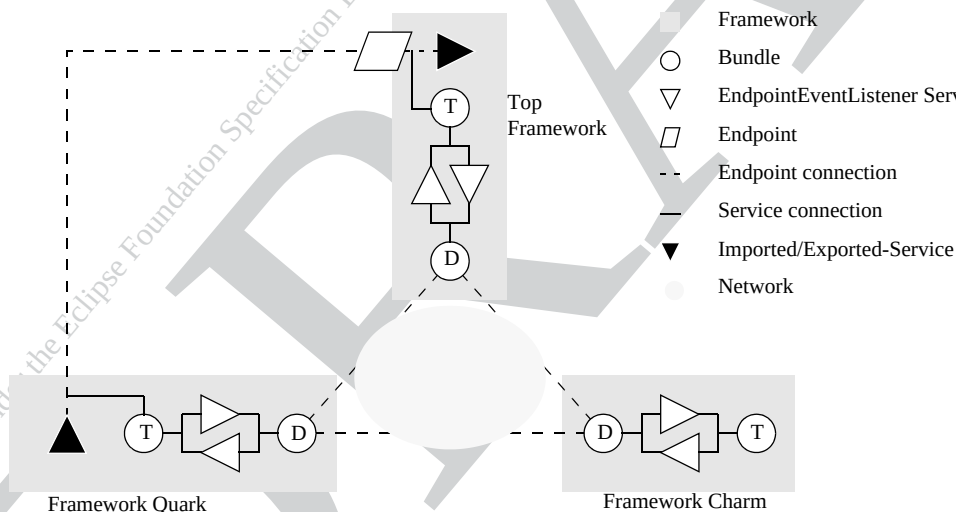
This architecture allows many different use cases. For example, a bundle could display a map of the topology by registering an Endpoint Event Listener with a scope for local Endpoints. Another example is the use of SLP to announce local Endpoints to a network and to discover remote Endpoints from other parties on this network.

An instance of this design is shown in Figure 122.5 on page 22. In this figure, there are 3 frameworks that collaborate through some discovery bundle. The Top framework has created an Endpoint and decides to notify all Endpoint Event Listeners and Endpoint Listeners registered in this framework that are scoped to this new Endpoint. Local bundle D has set its scope to all Endpoint Descriptions that originate from its local framework, it therefore receives the Endpoint Description from T. Bundle D then sends the Endpoint Description to all its peers on the network.

In the Quark framework, the manager bundle T has expressed an interest by setting its scope to a filter that matches the Endpoint Description from the Top framework. When the bundle D on the Quark framework receives the Endpoint Description from bundle D on the Top framework, it matches it against all local Endpoint Event Listener's scope. In this case, the local manager bundle T matches and is given the Endpoint Description. The manager then uses the Remote Service Admin service to import the exported service described by the given Endpoint Description.

Figure 122.5

Endpoint Discovery Architecture. T=Topology Manager, D=Discovery



The previous description is just one of the possible usages of the Endpoint Event Listener. For example, the discovery bundles could communicate the scopes to their peers. These peers could then register an Endpoint Event Listener per peer, minimizing the network traffic because Endpoint Descriptions do not have to be broadcast to all peers.

Another alternative usage is described in *Endpoint Description Extender Format* on page 27. In this chapter the extender pattern is used to retrieve Endpoint Descriptions from resources in locally active bundles.

122.6.1

Scope and Filters

An Endpoint Event Listener or Endpoint Listener service is registered with the `ENDPOINT_LISTENER_SCOPE` service property. This property, which is String+, must be set and must contain at least one filter. If there is not at least one filter, then that Endpoint Event Listener or Endpoint Listener must not receive any Endpoint Descriptions.

Each filter in the scope is applied against the properties of the Endpoint Description until one succeeds. Only if one succeeds is the Endpoint informed about the existence of an Endpoint.

The Endpoint Description is designed to reflect the properties of the imported service, there is therefore a correspondence with the filters that are used by bundles that are listening for service registrations. The purpose of this design is to match the filter available through Listener Hook services, see *On Demand* on page 25 .

However, the purpose of the filters is more generic than just this use case. It can also be used to specify the interest in local Endpoints or remote Endpoints. For example, Topology Managers are only interested in remote Endpoints while discoverers are only interested in local Endpoints. It is easy to discriminate between local and remote by filtering on the `endpoint.framework.uuid` property. Endpoint Descriptions contain the Universally Unique ID (UUID) of the originating framework. This UUID must be available from the local framework as well. See *Framework UUID* on page 15 .

122.6.2 Endpoint Event Listener Interface

The `EndpointEventListener` interface has the following method:

- `endpointChanged(EndpointEvent,String)` – Notify the Endpoint Event Listener of changes to an Endpoint. The change could entail the addition or removal of an Endpoint or the modification of the properties of an existing Endpoint. Multiple identical events should be counted as a single such event.

These methods must only be called if the Endpoint Event Listener service has a filter in its scope that matches the Endpoint Description properties.

The Endpoint Event Listener interface is *idempotent*. Endpoint Description Providers must inform an Endpoint Event Listener service (and its deprecated predecessor Endpoint Listener service) that is registered of all their matching Endpoints. The only way to find out about all available Endpoints is to register an Endpoint Event Listener (or Endpoint Listener) that is then informed by all available Endpoint Description Providers of their known Endpoint Descriptions that match their scope.

122.6.3 Endpoint Listener Interface

The `EndpointListener` interface is marked as Deprecated because the `EndpointEventListener` interface must be used in preference when both are implemented by the same object. The `EndpointEvent` interface has the following methods:

- `endpointAdded(EndpointDescription,String)` – Notify the Endpoint Listener of a new Endpoint Description. The second parameter is the filter that matched the Endpoint Description. Registering the same Endpoint multiple times counts as a single registration.
- `endpointRemoved(EndpointDescription,String)` – Notify the Endpoint Listener that the provided Endpoint Description is no longer available.

These methods must only be called if the Endpoint Listener service has a filter in its scope that matches the Endpoint Description properties. The reason for the filter string in the methods is to simplify and speed up matching an Endpoint Description to the cause of interest. For example, if the Listener Hook is used to do on demand import of services, then the filter can be associated with the Listener Info of the hook, see *On Demand* on page 25 . If multiple filters in the scope match the Endpoint Description then the first filter in the scope must be passed.

The Endpoint Listener interface is *idempotent*. Endpoint Description Providers must inform an Endpoint Listener service that is registered of all their matching Endpoints.

122.6.4 Endpoint Event Listener and Endpoint Listener Implementations

An Endpoint Event Listener service tracks the known Endpoints in its given scope. There are potentially a large number of bundles involved in creating this federated registry of Endpoints. To ensure

that no Endpoint Descriptions are orphaned or unnecessarily missed, an Endpoint Event Listener implementation must follow the following rules:

- *Registration* – The Endpoint Event Listener service is called with an event of type `ADDED` for all known Endpoint Descriptions that the bundles in the local framework are aware of. Similarly, Endpoint Listener services are called with an `endpointAdded(EndpointDescription,String)` method for all these.
- *Tracking providers* – An Endpoint Event Listener or Endpoint Listener must track the bundles that provide it with Endpoint Descriptions. If a bundle that provided Endpoint Descriptions is stopped, all Endpoint Descriptions that were provided by that bundle must be removed. This can be implemented straightforwardly with a Service Factory.
- *Scope modification* – An Endpoint Event Listener or Endpoint Listener is allowed to modify the set of filters in its scope through a service property modification. This modification must result in new and/or existing Endpoint Descriptions to be added, however, existing Endpoints that are no longer in scope are not required to be explicitly removed by their sources. It is the responsibility for the Endpoint Listener to remove these orphaned Endpoint Description from its view.
- *Endpoint mutability* – An Endpoint Description can change its Properties. The way this is handled is different for Endpoint Event Listeners and Endpoint Listeners. An Endpoint Event Listener receives a change event of type `MODIFIED` when the Properties of an existing Endpoint are modified. If the modification means that the Endpoint no longer matches the listener scope an event of type `MODIFIED_ENDMATCH` is sent instead. Endpoint Listener services receive a sequence of `endpointRemoved(EndpointDescription,String)` and `endpointAdded(EndpointDescription,String)` callbacks when the Properties of an Endpoint are modified.

Endpoint Descriptions can be added from different sources and providers of Endpoint Descriptions often use asynchronous and potentially unreliable communications. An implementation must therefore handle the addition of multiple equal Endpoint Descriptions from different sources as well as from the same source. Implementations must not count the number of registrations, a remove operation of an Endpoint Description is final for each source. That is, if source A added Endpoint Description e, then it can only be removed by source A. However, if source A added e multiple times, then it only needs to be removed once. Removals of Endpoint Descriptions that have not been added (or were removed before) should be ignored.

The discovery of Endpoints is a fundamentally indeterministic process and implementations of Endpoint Event Listener services should realize that there are no guarantees that an added Endpoint Description is always describing a valid Endpoint.

122.6.5

Endpoint Description Providers

The Endpoint Event Listener and Endpoint Listener services are based on an asynchronous, unreliable, best effort model because there are few guarantees in a distributed world. It is the task of an Endpoint Description Provider, for example a discovery bundle, to keep the Endpoint Event Listener services up to date of any Endpoint Descriptions the provider is aware of and that match the tracked service's scope.

If an Endpoint Event Listener or Endpoint Listener service is registered, a provider must add all matching Endpoint Descriptions that it is aware of and match the tracked listener's scope. This can be done during registration or asynchronously later. For example, it is possible to use the filters in the scope to request remote systems for any Endpoint Descriptions that match those filters. For expediency reasons, the service registration event should not be delayed until those results return; it is therefore applicable to add these Endpoint Descriptions later when the returns from the remote systems finally arrive.

If a tracked listener service object is advertised as both an Endpoint Event Listener and an Endpoint Listener then the EndpointDescription Provider must ignore the EndpointListener interface, and treat the listener only as an Endpoint Event Listener. Remote Service Admin 1.0 actors will be unaware of the EndpointEventListener interface, and will treat the service object purely as an Endpoint

Listener. This restriction ensures that all actors will treat the service either as an Endpoint Event Listener, or an Endpoint Listener, but never as both. As a result the listener service will not have to disambiguate duplicate events from a single source. If an Endpoint Description Provider uses both the Endpoint Listener and Endpoint Event Listener interfaces of a single service object then the resulting behavior is undefined. The implementation may detect the misuse and throw an Exception, process or ignore the events from one of the interfaces, or it may simply corrupt the internal registry of Endpoints within the listener.

A tracked Endpoint Event Listener or Endpoint Listener is allowed to modify its scope by setting new properties on its Service Registration. An Endpoint Description provider must process the new scope and add any newly matching Endpoint Descriptions. It is not necessary to remove any Endpoint Descriptions that were added before but no longer match the new scope. Removing those orphaned descriptions is the responsibility of the listener implementation.

It is not necessary to remove any registered Endpoint Descriptions when the Endpoint Event Listener or Endpoint Listener is unregistered; also here it is the responsibility of the listener to do the proper cleanup.

122.6.6

On Demand

A common distribution policy is to import services that are being listened for by local bundles. For example, when a bundle opens a Service Tracker on the Log Service, a Topology Manager could be notified and attempt to find a Log Service in the local cluster and then import this service in the local Service Registry.

The OSGi framework provides service hooks for exactly this purpose. A Topology Manager can register a Listener Hook service and receive the information about bundles that have specified an interests in specific services.

For example, a bundle creates the following Service Tracker:

```
ServiceTracker st = new ServiceTracker(context,
    LogService.class.getName() );
st.open();
```

This Service Tracker will register a Service Listener with the OSGi framework. This will cause the framework to add a ListenerInfo to any Listener Hook services. The getFilter method on a ListenerInfo object provides a filter that is directly applicable for the Endpoint Event Listener's scope. In the previous example, this would be the filter:

```
(objectClass=org.osgi.service.log.LogService)
```

A Topology Manager could verify if this listener is satisfied. That is, if it has at least one service. If no such service could be found, it could then add this filter to its Endpoint Event Listener's scope to detect remote implementations of this service. If such an Endpoint is detected, it could then request the import of this service through the Remote Service Admin service.

122.7

Events

The Remote Service Admin service must synchronously inform any Remote Service Admin Listener services of events as they happen. Client of the events should return quickly and not perform any but trivial processing in the same thread.

The following event types are defined:

- **EXPORT_ERROR** - An exported service has run into an unrecoverable error, although the Export Registration has not been closed yet. The event carries the Export Registration as well as the Exception that caused the problem, if present.

- **EXPORT_REGISTRATION** - The Remote Service Admin has registered a new Export Registration.
- **EXPORT_UNREGISTRATION** - An Export Registration has been closed, the service is no longer exported and the Endpoint is no longer active when this was the last registration for that service/Endpoint combination.
- **EXPORT_UPDATE** - An exported service is updated. The service properties have changed.
- **EXPORT_WARNING** - An exported service is experiencing problems but the Endpoint is still available.
- **IMPORT_ERROR** - An imported service has run into a fatal error and has been shut down. The Import Registration should be closed by the Topology Manager that created them.
- **IMPORT_REGISTRATION** - A new Import Registration was created for a potentially existing service/Endpoint combination.
- **IMPORT_UNREGISTRATION** - An Import Registration was closed, removing the proxy if this was the last registration.
- **IMPORT_UPDATE** - An imported service is updated. The service properties have changed.
- **IMPORT_WARNING** - An imported service is experiencing problems but can continue to function.

The following properties are available on the event:

- **getType()** - The type of the event.
- **getException()** - Any exception, if present.
- **getExportReference()** - An export reference, if applicable.
- **getImportReference()** - An import reference, if applicable.
- **getSource()** - The source of the event, the Remote Service Admin service.

122.7.1

Event Admin Mapping

All Remote Service Admin events must be posted, which is asynchronously, to the Event Admin service, if present, under the following topic:

`org.osgi/service/remoteserviceadmin/<type>`

Where <type> represents the type of the event, for example `IMPORT_ERROR`.

The Event Admin event must have the following properties:

- **bundle** - (Bundle) The Remote Service Admin bundle.
- **bundle.id** - (Long) The id of the Remote Service Admin bundle.
- **bundle.symbolicname** - (String) The Bundle Symbolic Name of the Remote Service Admin.
- **bundle.version** - (Version) The version of the Remote Service Admin bundle.
- **bundle.signer** - (String[]) The signer of the Remote Service Admin bundle.
- **exception** - (Throwable) The Exception, if present. Also reported on the cause property for backward compatibility.
- **exception.class** - (String) The fully-qualified class name of the attached Exception.
- **exception.message** - (String) The message of the attached exception. Only set if the Exception message is not null.
- **endpoint.service.id** - (Long) Remote service id, if present.
- **endpoint.framework.uuid** - (String) Remote service's Framework UUID, if present.
- **endpoint.id** - (String) The id of the Endpoint, if present.
- **objectClass** - (String[]) The interface names, if present.
- **service.imported.configs** - (String+) The configuration types of the imported service, if present.
- **timestamp** - (Long) The time when the event occurred.

- event - (RemoteServiceAdminEvent) The RemoteServiceAdminEvent object that caused this event.

122.8 Endpoint Description Extender Format

The Endpoint Description Extender format is a possibility to deliver Endpoint Descriptions in bundles. This section defines an XML schema and how to locate XML definition resources that use this schema to define Endpoint Descriptions. The definition resource is a simple property based model that can define the same information as the properties on an imported service. If a bundle with the description is *ready* (ACTIVE or lazy activation and in the STARTING state), then this static description can be disseminated through the Endpoint Event Listeners that have specified an interest in this description. If the bundle is stopped, the corresponding Endpoints must be removed.

XML documents containing remote service descriptions must be specified by the Remote-Service header in the manifest. The structure of the Remote Service header is:

Remote-Service ::= header // See Common Header Syntax in Core

The value of the header is a comma separated list of paths. A path is:

- A directory if it ends with a solidus ('/' \u002F). A directory is scanned for *.xml files.
- A path with wildcards. Such a path can use the wildcards in its last component, as defined in the findEntries method.
- A complete path, not having wildcards not ending in a solidus ('/' \u002F).

The Remote-Service header has no architected directives or attributes, unrecognized attributes and directives must be ignored.

A Remote-Service manifest header specified in a fragment must be ignored. However, XML documents referenced by a bundle's Remote-Service manifest header can be contained in attached fragments. The required behavior for this is implemented in the findEntries method.

The extender must process each XML document specified in this header. If an XML document specified by the header cannot be located in the bundle and its attached fragments, the extender must log an error message with the Log Service, if present, and continue.

For example:

Remote-Service: lib/, remote/osgi/*.dsc, cnf/google.xml

This matches all resources in the lib directory matching *.xml, all resources in the /remote/osgi directory that end with .dsc, as well as the google.xml resource in the cnf directory.

The namespace of these XML resources must be:

<http://www.osgi.org/xmlns/rsa/v1.0.0>

This namespace describes a set of Endpoint Descriptions, where each Endpoint Description can provide a set of properties. The structure of this schema is:

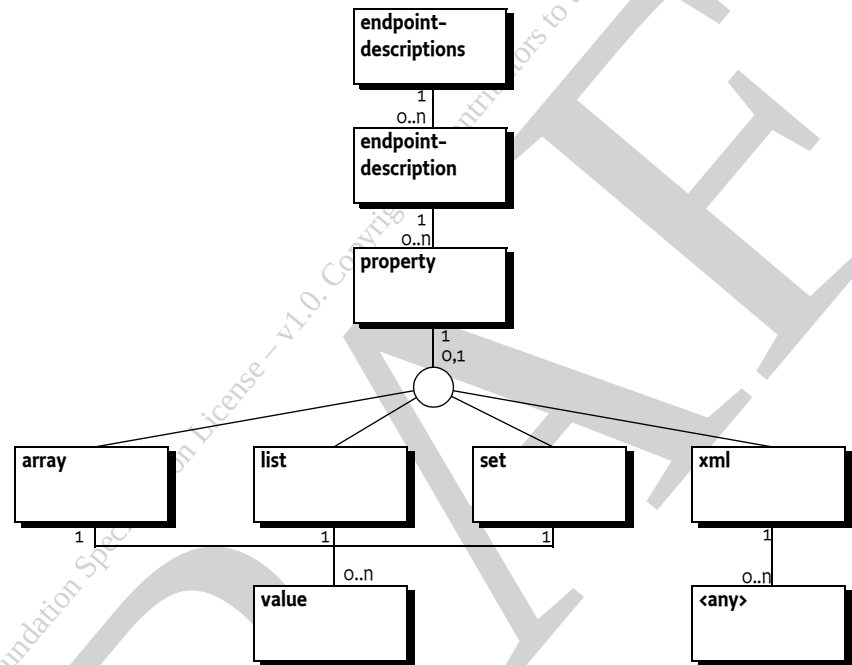
```

endpoint-descriptions ::= <endpoint-description>*
endpoint-description ::= <property>*
property               ::= ( <array> | <list> | <set> | <xml> )?
array                  ::= <value> *
list                   ::= <value> *
```

set ::= <value> *
xml ::= <*> *

This structure is depicted in Figure 122.6 on page 28 .

Figure 122.6 Endpoint Description XML Structure



The property element has the attributes listed in table Table 122.2 .

Table 122.2 Property Attributes

Attribute	Type	Description
name	String	The required name of the property. The type maps to the XML Schema xsd:string type.

Attribute	Type	Description
value-type	String long Long double Double float Float int Integer byte Byte char Character boolean Boolean short Short	The optional type name of the property, the default is String. Any value in the value attribute or the value element when collections are used must be converted to the corresponding Java types. If the primitive form, for example byte, is specified for non-array types, then the value must be silently converted to the corresponding wrapper type.
value	String	The value. Must be converted to the specified type if this is not the String type. The value attribute must not be used when the property element has a child element.

A property can have an array, list, set, or xml child element. If a child element is present then it is an error if the value attribute is defined. It is also an error if there is no child element and no value attribute.

The array, list, or set are *multi-valued*. That is, they contain 0 or more value elements. A value element contains text (a string) that must be converted to the given value-type or if not specified, left as is. Conversion must *trim* the leading and trailing white space characters as defined in the Character.isWhitespace method. No trimming must be done for strings. An array of primitive integers like `int[] {1,42,97}` can be encoded as follows:

```
<property name="integers" value-type="int">
  <array>
    <value> 1</value>
    <value>42</value>
    <value>97</value>
  </array>
</property>
```

The xml element is used to convey XML from other namespaces, it is allowed to contain one foreign XML root element, with any number of children, that will act as the root element of an XML document. This root element will be included in the corresponding property as a string. The XML element must be a valid XML document but not contain the XML processing instructions, the part between the `<?` and `?>`. The value-type of the property must be String or not set when an xml element is used, using another type is invalid.

The xml element can be used to embed configuration information, making the Endpoint Description self contained.

The following is an example of an endpoint-descriptions resource.

```
<?xml version="1.0" encoding="UTF-8"?>
<endpoint-descriptions xmlns="http://www.osgi.org/xmlns/rsa/v1.0.0">
  <endpoint-description>
    <property name="service.intents">
      <list>
        <value>SOAP</value>
        <value>HTTP</value>
      </list>
    </property>
    <property name="endpoint.id" value="http://ws.acme.com:9000/hello"/>
    <property name="endpoint.package.version.com.acme" value="4.2"/>
    <property name="objectClass">
      <array>
        <value>com.acme.Foo</value>
      </array>
    </property>
    <property name="service.imported.configs" value="com.acme"/>
    <property name="com.acme.ws.xml">
      <xml>
        <config xmlns="http://acme.com/defs">
          <port>1029</port>
          <host>www.acme.com</host>
        </config>
      </xml>
    </property>
  </endpoint-description>
</endpoint-descriptions>
```

Besides being in a separate resource, the static configuration as described here could also be part of a larger XML file. In that case the parser must ignore elements not part of the `http://www.osgi.org/xmlns/rsa/v1.0.0` namespace schema.

122.8.1 XML Schema

This namespace of the schema is:

`http://www.osgi.org/xmlns/rsa/v1.0.0`

```
<?xml version="1.0" encoding="UTF-8"?>
<!--
  Copyright (c) Contributors to the Eclipse Foundation

  Licensed under the Apache License, Version 2.0 (the "License");
  you may not use this file except in compliance with the License.
  You may obtain a copy of the License at
```

`http://www.apache.org/licenses/LICENSE-2.0`

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

SPDX-License-Identifier: Apache-2.0

```
-->
```

```
<schema xmlns="http://www.w3.org/2001/XMLSchema"
  xmlns:rsa="http://www.osgi.org/xmlns/rsa/v1.0.0"
  targetNamespace="http://www.osgi.org/xmlns/rsa/v1.0.0"
  elementFormDefault="qualified" version="1.0.1">
```

```

<annotation>
  <documentation xml:lang="en">
    This is the XML Schema for endpoint descriptions used by
    the Remote Service Admin Specification. Endpoint descriptions
    are used to describe remote services to a client in cases
    where a real live Discovery system isn't used. An extender,
    such as a local Discovery Service can look for service
    descriptions in installed bundles and inform the Topology
    Manager of these remote services. The Topology Manager can then
    instruct the Remote Service Admin to create client proxies for
    these services.
  </documentation>
</annotation>

<element name="endpoint-descriptions" type="rsa:Tendpoint-descriptions" />

<complexType name="Tendpoint-descriptions">
  <sequence>
    <element name="endpoint-description" type="rsa:Tendpoint-description"
      minOccurs="1" maxOccurs="unbounded" />

    <!--
      It is non-deterministic, per W3C XML Schema 1.0:
      http://www.w3.org/TR/xmlschema-1/#cos-nonambig to use
      namespace="##any" below.
    -->
    <any namespace="##other" minOccurs="0" maxOccurs="unbounded"
      processContents="lax" />
  </sequence>
  <anyAttribute processContents="lax" />
</complexType>

<complexType name="Tendpoint-description">
  <annotation>
    <documentation xml:lang="en">
      A Distribution Provider can register a proxy with the properties
      provided. Whether or not it is instructed to do so, is up to the
      Topology Manager. If any 'intents' properties are specified then the
      Distribution Provider should only register a proxy if it can support
      those intents.
    </documentation>
  </annotation>
  <sequence>
    <element name="property" type="rsa:Tproperty" minOccurs="1"
      maxOccurs="unbounded" />
    <any namespace="##other" minOccurs="0" maxOccurs="unbounded"
      processContents="lax" />
  </sequence>
  <anyAttribute processContents="lax" />
</complexType>

<complexType name="Tproperty" mixed="true">
  <sequence>
    <choice minOccurs="0" maxOccurs="1">
      <element name="array" type="rsa:Tmulti-value"/>
      <element name="list" type="rsa:Tmulti-value"/>
      <element name="set" type="rsa:Tmulti-value"/>
      <element name="xml" type="rsa:Txml"/>
    </choice>
    <any namespace="##other" minOccurs="0" maxOccurs="unbounded"
      processContents="lax" />
  </sequence>
  <attribute name="name" type="string" use="required" />
  <attribute name="value" type="string" use="optional" />
  <attribute name="value-type" type="rsa:Tvalue-types" default="String" use="optional" />
  <anyAttribute processContents="lax" />
</complexType>

<complexType name="Tmulti-value">
  <sequence>
    <element name="value" minOccurs="0" maxOccurs="unbounded" type="rsa:Tvalue"/>
    <any namespace="##other" minOccurs="0" maxOccurs="unbounded"
      processContents="lax" />
  </sequence>
  <anyAttribute processContents="lax" />

```

```

</complexType>

<complexType name="Tvalue" mixed="true">
  <sequence>
    <element name="xml" minOccurs="0" maxOccurs="1" type="rsa:Txml"/>
    <any namespace="##other" minOccurs="0" maxOccurs="unbounded"
      processContents="lax" />
  </sequence>
  <anyAttribute processContents="lax" />
</complexType>

<!-- Specifies the data type of a property or of the elements in a multi-value
property. Numerical and boolean values are trimmed before they are processed.
Simple types are automatically boxed if needed. Only the array data type
allows for simple type values. When specifying a simple type on any other
type of property it will automatically be boxed. -->
<simpleType name="Tvalue-types">
  <restriction base="string">
    <enumeration value="String" />
    <enumeration value="long" />
    <enumeration value="Long" />
    <enumeration value="double" />
    <enumeration value="Double" />
    <enumeration value="float" />
    <enumeration value="Float" />
    <enumeration value="int" />
    <enumeration value="Integer" />
    <enumeration value="byte" />
    <enumeration value="Byte" />
    <enumeration value="char" />
    <enumeration value="Character" />
    <enumeration value="boolean" />
    <enumeration value="Boolean" />
    <enumeration value="short" />
    <enumeration value="Short" />
  </restriction>
</simpleType>

<!-- This complex type allows literal XML to be specified in an <xml/> tag (which
is more convenient than putting it in a CDATA section).
The embedded XML must be well-formed and not be in the rsa namespace. It will
be put in a String value of a property or in an element of a multi-value
property of base type String. The XML will be prefixed with the standard
<?XML ?> header which is copied from the enclosing document. Hence it will
carry the same version and encoding as the document in the rsa namespace. -->
<complexType name="Txml">
  <sequence>
    <any namespace="##other" minOccurs="1" maxOccurs="1"
      processContents="lax" />
  </sequence>
  <anyAttribute processContents="lax" />
</complexType>

<attribute name="must-understand" type="boolean" default="false">
  <annotation>
    <documentation xml:lang="en">
      This attribute should be used by extensions to documents
      to require that the document consumer understand the
      extension.
    </documentation>
  </annotation>
</attribute>
</schema>

```

122.9 Capability Namespaces

122.9.1 Local Discovery Extender

A bundle containing Endpoint Description Extender resources can indicate its dependency on the Remote Service Admin extender by declaring a requirement on the `osgi.extender.namespace`.

```
Require-Capability: osgi.extender;
    filter:="(&(osgi.extender=osgi.remoteserviceadmin.localdiscovery)
    (version>=1.0) (! (version>=2.0)))"
```

With this constraint declared a bundle that depends on the extender will fail to resolve if no extender is present in the framework.

Implementations of this specification must provide this extender capability at version 1.0 as follows:

```
Provide-Capability: osgi.extender;
    osgi.extender="osgi.remoteserviceadmin.localdiscovery";
    version:Version="1.0";
    uses:="org.osgi.service.remoteserviceadmin"
```

The reason that the extender capability is declared at version 1.0 is because the extender is unchanged from version 1.0 of this specification.

122.9.2 Discovery Provider Capability

Discovery Providers use the `osgi.remoteserviceadmin.discovery` namespace to declare themselves as such. The version defined for this namespace indicates the version of this specification that the discovery provider supports.

This namespace has a defined attribute, `protocols` of type `List<String>`, which contains a list of the discovery protocols supported by the discovery provider. Local discovery providers (using the *Endpoint Description Extender Format* on page 27), should use the value `local` to indicate that they support this. Additionally, it defines a `version` attribute. Other values for the `protocols` attribute are implementation specific.

Table 122.3 *osgi.remoteserviceadmin.discovery Namespace*

Name	Kind	M/O	Type	Syntax	Description
protocols	CA	M	List<String>	symbolic-name	The discovery protocols supported. A value of <code>local</code> indicates support for the <i>Endpoint Description Extender Format</i> on page 27 .
version	CA	M	Version	version	This version must correspond to the version of the Remote Service Admin specification.

Example: A discovery provider that provides local and SLP discovery:

```
Provide-Capability: osgi.remoteserviceadmin.discovery;
    protocols:List<String>="SLP,local"; version:Version=1.1
```

122.9.3 Distribution Provider Capability

Distribution providers advertise their supported distribution mechanisms using configuration types. These are selected at runtime using the `service.exported.configs` service property. Distribution providers can use the `osgi.remoteserviceadmin.distribution` namespace with attribute `configs`, of type `List<String>`, to advertise the supported config types.

Table 122.4 *osgi.remoteserviceadmin.distribution Namespace*

Name	Kind	M/O	Type	Syntax	Description
configs	CA	M	List<String>	symbolic-name	Supported configuration types. See <i>Endpoint Description</i> on page 12.
version	CA	M	Version	version	This version must correspond to the version of the Remote Service Admin specification.

Example: A Distribution provider that supports the org.acme.jaxws and org.acme.jaxrs configuration types:

```
Provide-Capability: osgi.remoteserviceadmin.distribution;
  configs:List<String>="org.acme.jaxws,org.acme.jaxrs"; version:Version=1.1
```

122.9.4

Topology Manager Capability

Remote Service Admin topology managers may use different policies when determining which services to export and/or import. Topology managers use the namespace `osgi.remoteserviceadmin.topology` to declare this behavior. This namespace defines the policy attribute of type `List<String>`. Values are implementation specific, but example definitions can be found at *Example Use Cases* on page 11.

Table 122.5 *osgi.remoteserviceadmin.topology Namespace*

Name	Kind	M/O	Type	Syntax	Description
policy	CA	M	List<String>	symbolic-name	The policy used for importing and exporting services. In general the policy is implementation specific.
version	CA	M	Version	version	This version must correspond to the version of the Remote Service Admin specification.

Example: A Topology manager that supports a *promiscuous* policy:

```
Provide-Capability: osgi.remoteserviceadmin.topology;
  policy:List<String>=promiscuous; version:Version=1.1
```

122.9.5

Service Capability

The Distribution Provider provides the *Remote Service Admin* service. To inform tools about this service it must provide the `osgi.service` namespace representing the [RemoteServiceAdmin](#) service. This capability must also declare a uses constraint for the `org.osgi.service.remoteserviceadmin` package:

```
Provide-Capability: osgi.service;
  objectClass:List<String>=
    "org.osgi.service.remoteserviceadmin.RemoteServiceAdmin";
  uses:="org.osgi.service.remoteserviceadmin"
```

This capability must follow the rules defined for the ???.

122.10

Advice to implementations

This section is not intended to be normative, but offers advice to implementations as to how the complexity of supporting both the new Endpoint Event Listener and Endpoint Listener services can be managed and minimized. This advice applies to both Discovery Providers and Topology Managers implementing Remote Service Admin 1.1.

122.10.1 Notifying listeners

Endpoint Event Listeners and Endpoint Listeners have a very similar behavior and lifecycle. They also use the same property names to define their scope filter. It is therefore relatively simple for an Endpoint Description Provider to notify both Endpoint Listener and Endpoint Event Listeners using a single code path.

One possible mechanism is to track both the listener types using the same Service Tracker. If the tracked Service Reference advertises the `EndpointEventListener` interface then it must be treated as an Endpoint Event Listener. If not then the Endpoint Listener service can be wrapped in an adapter that converts Endpoint Event Listener events into the appropriate Endpoint Listener calls. The main notification code path can then treat every listener as an Endpoint Event Listener.

122.10.2 Receiving Endpoint lifecycle notifications

The Remote Service Admin 1.1 specification is backward compatible with version 1.0, meaning that version 1.1 actors must register an Endpoint Listener service. There is no restriction requiring this listener to be the same service as the Endpoint Event Listener, however there is a significant advantage to combining the listeners into a single service registration.

By making the two listeners a single service object a bundle can guarantee that it will not receive multiple notifications for the same event. If the service registrations are separate then Endpoint Description Providers will see two separate listeners, and notify them both. As a single service registration only one event will occur, and using the highest mutually supported version of the Remote Service Admin Specification.

122.11 Security

From a security point of view distribution is a significant threat. A Distribution Provider requires very significant capabilities to be able to proxy services. In many situations it will be required to grant the distribution provider All Permission. It is therefore highly recommended that Distribution Providers use trusted links and ensure that it is not possible to attack a system through the Remote Services Admin service and used discovery protocols.

122.11.1 Import and Export Registrations

Import and Export Registrations are *capabilities*. That is, they can only be obtained when the caller has the proper permissions but once obtained they are no longer checked. The caller should therefore be careful to share those objects with other bundles. Export and Import References are free to share.

122.11.2 Endpoint Permission

The Remote Service Admin implementation requires a large set of permissions because it must be able to distribute potentially any service. Giving these extensive capabilities to all Topology Managers would make it harder to developer general Topology Managers that implements specific scenarios. For this reason, this specification provides an Endpoint Permission.

When an Endpoint Permission must be verified, it must be created with an Endpoint Description as argument, like:

```
sm.checkPermission( new EndpointPermission(anEndpoint, localUUID, READ));
```

The standard name and action constructor is used to define a permission. The name argument is a filter expression. The filter for an Endpoint Permission is applied to the properties of an Endpoint Description. The localUUID must map to the UUID of the framework of the caller of this constructor, see *Framework UUID* on page 15 . This localUUID is used to allow a the permissions to use the `<<LOCAL>>` magic name in the permission filter to refer to the local framework.

The filter expression can use the following magic value:

- <<LOCAL>> - This value represents the framework UUID of the framework that this bundle belongs to. The following example restricts the visibility to descriptions of local Endpoints:

```
ALLOW {
  ... EndpointPermission
    "(endpoint.framework.uuid=<<LOCAL>>)"
    "READ" }
```

An Endpoint Permission that has the actions listed in the following table.

Table 122.6 Endpoint Permission Actions

Action	Methods	Description
IMPORT	<code>importService(EndpointDescription)</code>	Import an Endpoint
EXPORT	<code>exportService(ServiceReference,Map)</code>	Export a service
READ	<code>getExportedServices()</code> <code>getImportedEndpoints()</code> <code>remoteAdminEvent(RemoteServiceAdminEvent)</code>	See the presence of distributed services. The IMPORT and EXPORT action imply READ. Distribution of events to the Remote Service Admin Listener. The Remote Service Admin must verify that the listener's bundle has the proper permission. No events should be delivered that are not implied.

122.12 org.osgi.service.remoteserviceadmin

Remote Service Admin Package Version 1.1.

Bundles wishing to use this package must list the package in the Import-Package header of the bundle's manifest. This package has two types of users: the consumers that use the API in this package and the providers that implement the API in this package.

Example import for consumers using the API in this package:

```
{@code Import-Package: org.osgi.service.remoteserviceadmin; version="[1.1,2.0)"}
```

Example import for providers implementing the API in this package:

```
{@code Import-Package: org.osgi.service.remoteserviceadmin; version="[1.1,1.2)"}
```

122.12.1 Summary

- `EndpointDescription` - A description of an endpoint that provides sufficient information for a compatible distribution provider to create a connection to this endpoint An Endpoint Description is easy to transfer between different systems because it is property based where the property keys are strings and the values are simple types.
- `EndpointEvent` - An Endpoint Event.
- `EndpointEventListener` - A white board service that represents a listener for endpoints.
- `EndpointListener` - Deprecated white board service that represents a listener for endpoints.
- `EndpointPermission` - A bundle's authority to export, import or read an Endpoint.
- `ExportReference` - An Export Reference associates a service with a local endpoint.
- `ExportRegistration` - An Export Registration associates a service to a local endpoint.

- `ImportReference` - An Import Reference associates an active proxy service to a remote endpoint.
- `ImportRegistration` - An Import Registration associates an active proxy service to a remote endpoint.
- `RemoteConstants` - Provide the definition of the constants used in the Remote Service Admin specification.
- `RemoteServiceAdmin` - A Remote Service Admin manages the import and export of services.
- `RemoteServiceAdminEvent` - Provides the event information for a Remote Service Admin event.
- `RemoteServiceAdminListener` - A {@link RemoteServiceAdminEvent} listener is notified synchronously of any export or import registrations and unregistrations.

122.12.2 public class EndpointDescription

A description of an endpoint that provides sufficient information for a compatible distribution provider to create a connection to this endpoint. An Endpoint Description is easy to transfer between different systems because it is property based where the property keys are strings and the values are simple types. This allows it to be used as a communications device to convey available endpoint information to nodes in a network. An Endpoint Description reflects the perspective of an *importer*. That is, the property keys have been chosen to match filters that are created by client bundles that need a service. Therefore the map must not contain any {@code service.exported.*} property and must contain the corresponding {@code service.imported.*} ones. The {@code service.intents} property must contain the intents provided by the service itself combined with the intents added by the exporting distribution provider. Qualified intents appear fully expanded on this property.

Concurrency Immutable

122.12.2.1 public EndpointDescription(Map<String, ?> properties)

properties The map from which to create the Endpoint Description. The keys in the map must be type {@code String} and, since the keys are case insensitive, there must be no duplicates with case variation.

- Create an Endpoint Description from a Map.

The {@link RemoteConstants#ENDPOINT_ID endpoint.id}, {@link RemoteConstants#SERVICE_IMPORTED_CONFIGS service.imported.configs} and {@code objectClass} properties must be set.

Throws `IllegalArgumentException` - When the properties are not proper for an Endpoint Description.

122.12.2.2 public EndpointDescription(ServiceReference<?> reference, Map<String, ?> properties)

reference A service reference that can be exported.

properties Map of properties. This argument can be {@code null}. The keys in the map must be type {@code String} and, since the keys are case insensitive, there must be no duplicates with case variation.

- Create an Endpoint Description based on a Service Reference and a Map of properties. The properties in the map take precedence over the properties in the Service Reference.

This method will automatically set the {@link RemoteConstants#ENDPOINT_FRAMEWORK_UUID endpoint.framework.uuid} and {@link RemoteConstants#ENDPOINT_SERVICE_ID endpoint.service.id} properties based on the specified Service Reference as well as the {@link RemoteConstants#SERVICE_IMPORTED service.imported} property if they are not specified as properties.

The {@link RemoteConstants#ENDPOINT_ID endpoint.id}, {@link RemoteConstants#SERVICE_IMPORTED_CONFIGS service.imported.configs} and {@code objectClass} properties must be set.

Throws `IllegalArgumentException` - When the properties are not proper for an Endpoint Description

122.12.2.3 public boolean equals(Object other)

other The {@code EndpointDescription} object to be compared.

- Compares this {@code EndpointDescription} object to another object.

An Endpoint Description is considered to be **equal to** another Endpoint Description if their ids are equal.

Returns {@code true} if {@code object} is a {@code EndpointDescription} and is equal to this object; {@code false} otherwise.

122.12.2.4 **public List<String> getConfigurationTypes()**

- Returns the configuration types. A distribution provider exports a service with an endpoint. This endpoint uses some kind of communications protocol with a set of configuration parameters. There are many different types but each endpoint is configured by only one configuration type. However, a distribution provider can be aware of different configuration types and provide synonyms to increase the change a receiving distribution provider can create a connection to this endpoint. This value of the configuration types is stored in the {@link RemoteConstants#SERVICE_IMPORTED_CONFIGS} service property.

Returns An unmodifiable list of the configuration types used for the associated endpoint and optionally synonyms.

122.12.2.5 **public String getFrameworkUUID()**

- Return the framework UUID for the remote service, if present. The value of the remote framework UUID is stored in the {@link RemoteConstants#ENDPOINT_FRAMEWORK_UUID} endpoint property.

Returns Remote Framework UUID, or {@code null} if this endpoint is not associated with an OSGi framework having a framework UUID.

122.12.2.6 **public String getId()**

- Returns the endpoint's id. The id is an opaque id for an endpoint. No two different endpoints must have the same id. Two Endpoint Descriptions with the same id must represent the same endpoint. The value of the id is stored in the {@link RemoteConstants#ENDPOINT_ID} property.

Returns The id of the endpoint, never {@code null}. The returned value has leading and trailing whitespace removed.

122.12.2.7 **public List<String> getIntents()**

- Return the list of intents implemented by this endpoint. The intents are based on the service.intents on an imported service, except for any intents that are additionally provided by the importing distribution provider. All qualified intents must have been expanded. This value of the intents is stored in the {@link RemoteConstants#SERVICE_INTENTS} service property.

Returns An unmodifiable list of expanded intents that are provided by this endpoint.

122.12.2.8 **public List<String> getInterfaces()**

- Provide the list of interfaces implemented by the exported service. The value of the interfaces is derived from the {@code objectClass} property.

Returns An unmodifiable list of Java interface names implemented by this endpoint.

122.12.2.9 **public Version getPackageVersion(String packageName)**

packageName The name of the package for which a version is requested.

- Provide the version of the given package name. The version is encoded by prefixing the given package name with {@link RemoteConstants#ENDPOINT_PACKAGE_VERSION_} endpoint.package.version., and then using this as an endpoint property key. For example:

endpoint.package.version.com.acme

The value of this property is in String format and will be converted to a {@code Version} object by this method.

Returns The version of the specified package or {@code Version.emptyVersion} if the package has no version in this Endpoint Description.

Throws `IllegalArgumentException` – If the version property value is not String.

122.12.2.10 **public Map<String, Object> getProperties()**

- Returns all endpoint properties.

Returns An unmodifiable map referring to the properties of this Endpoint Description.

122.12.2.11 **public long getServiceId()**

- Returns the service id for the service exported through this endpoint. This is the service id under which the framework has registered the service. This field together with the Framework UUID is a globally unique id for a service. The value of the remote service id is stored in the {@link RemoteConstants#ENDPOINT_SERVICE_ID} endpoint property.

Returns Service id of a service or 0 if this Endpoint Description does not relate to an OSGi service.

122.12.2.12 **public int hashCode()**

- Returns a hash code value for the object.

Returns An integer which is a hash code value for this object.

122.12.2.13 **public boolean isSameService(EndpointDescription other)**

other The Endpoint Description to look at

- Answers if this Endpoint Description refers to the same service instance as the given Endpoint Description. Two Endpoint Descriptions point to the same service if they have the same id or their framework UUIDs and remote service ids are equal.

Returns True if this endpoint description points to the same service as the other

122.12.2.14 **public boolean matches(String filter)**

filter The filter to test.

- Tests the properties of this {@code EndpointDescription} against the given filter using a case insensitive match.

Returns {@code true} If the properties of this {@code EndpointDescription} match the filter, {@code false} otherwise.

Throws `IllegalArgumentException` – If {@code filter} contains an invalid filter string that cannot be parsed.

122.12.2.15 **public String toString()**

- Returns the string representation of this EndpointDescription.

Returns String form of this EndpointDescription.

122.12.3 **public class EndpointEvent**

An Endpoint Event.

{@code EndpointEvent} objects are delivered to all registered {@link EndpointEventListener} services where the {@link EndpointDescription} properties match one of the filters specified in the {@link EndpointEventListener#ENDPOINT_LISTENER_SCOPE} registration properties of the Endpoint Event Listener.

A type code is used to identify the type of event. The following event types are defined:

- {@link #ADDED}
- {@link #REMOVED}
- {@link #MODIFIED}
- {@link #MODIFIED_ENDMATCH}

Additional event types may be defined in the future.

See Also EndpointEventListener

Since @since 1.1

Concurrency Immutable

122.12.3.1 **public static final int ADDED = 1**

An endpoint has been added.

This {@code EndpointEvent} type indicates that a new endpoint has been added. The endpoint is represented by the associated {@link EndpointDescription} object.

122.12.3.2 **public static final int MODIFIED = 4**

The properties of an endpoint have been modified.

This {@code EndpointEvent} type indicates that the properties of an existing endpoint have been modified. The endpoint is represented by the associated {@link EndpointDescription} object and its properties can be obtained via {@link EndpointDescription#getProperties()}. The endpoint properties still match the filters as specified in the {@link EndpointEventListener#ENDPOINT_LISTENER_SCOPE} filter.

122.12.3.3 **public static final int MODIFIED_ENDMATCH = 8**

The properties of an endpoint have been modified and the new properties no longer match the listener's filter.

This {@code EndpointEvent} type indicates that the properties of an existing endpoint have been modified and no longer match the filter. The endpoint is represented by the associated {@link EndpointDescription} object and its properties can be obtained via {@link EndpointDescription#getProperties()}. As a consequence of the modification the filters as specified in the {@link EndpointEventListener#ENDPOINT_LISTENER_SCOPE} do not match any more.

122.12.3.4 **public static final int REMOVED = 2**

An endpoint has been removed.

This {@code EndpointEvent} type indicates that an endpoint has been removed. The endpoint is represented by the associated {@link EndpointDescription} object.

122.12.3.5 **public EndpointEvent(int type, EndpointDescription endpoint)**

type The event type. See {@link #getType()}.

endpoint The endpoint associated with the event.

- Constructs a {@code EndpointEvent} object from the given arguments.

122.12.3.6 **public EndpointDescription getEndpoint()**

- Return the endpoint associated with this event.

Returns The endpoint associated with the event.

122.12.3.7**public int getType()**

- Return the type of this event.

The type values are:

- {@link #ADDED}
- {@link #REMOVED}
- {@link #MODIFIED}
- {@link #MODIFIED_ENDMATCH}

Returns The type of this event.

122.12.4**public abstract EndpointEventListener**

A white board service that represents a listener for endpoints. An Endpoint Event Listener represents a participant in the distributed model that is interested in Endpoint Descriptions. This white board service can be used in many different scenarios. However, the primary use case is to allow a remote manager to be informed of Endpoint Descriptions available in the network and inform the network about available Endpoint Descriptions. Both the network bundle and the manager bundle register an Endpoint Event Listener service. The manager informs the network bundle about Endpoints that it creates. The network bundles then uses a protocol like SLP to announce these local end-points to the network. If the network bundle discovers a new Endpoint through its discovery protocol, then it sends an Endpoint Description to all the Endpoint Listener services that are registered (except its own) that have specified an interest in that endpoint. Endpoint Event Listener services can express their *scope* with the service property {@link #ENDPOINT_LISTENER_SCOPE}. This service property is a list of filters. An Endpoint Description should only be given to a Endpoint Event Listener when there is at least one filter that matches the Endpoint Description properties. This filter model is quite flexible. For example, a discovery bundle is only interested in locally originating Endpoint Descriptions. The following filter ensures that it only sees local endpoints.

```
(org.osgi.framework.uid=72dc5fd9-5f8f-4f8f-9821-9ebb433a5b72)
```

In the same vein, a manager that is only interested in remote Endpoint Descriptions can use a filter like:

```
(!(org.osgi.framework.uid=72dc5fd9-5f8f-4f8f-9821-9ebb433a5b72))
```

Where in both cases, the given UUID is the UUID of the local framework that can be found in the Framework properties. The Endpoint Event Listener's scope maps very well to the service hooks. A manager can just register all filters found from the Listener Hook as its scope. This will automatically provide it with all known endpoints that match the given scope, without having to inspect the filter string. In general, when an Endpoint Description is discovered, it should be dispatched to all registered Endpoint Event Listener services. If a new Endpoint Event Listener is registered, it should be informed about all currently known Endpoints that match its scope. If a getter of the Endpoint Listener service is unregistered, then all its registered Endpoint Description objects must be removed. The Endpoint Event Listener models a *best effort* approach. Participating bundles should do their utmost to keep the listeners up to date, but implementers should realize that many endpoints come through unreliable discovery processes. The Endpoint Event Listener supersedes the {@link EndpointListener} interface as it also supports notifications around modifications of endpoints.

Since @since 1.1

Concurrency Thread-safe

122.12.4.1**public static final String ENDPOINT_LISTENER_SCOPE = "endpoint.listener.scope"**

Specifies the interest of this listener with filters. This listener is only interested in Endpoint Descriptions where its properties match the given filter. The type of this property must be {@code String+}.

122.12.4.2 public abstract void endpointChanged(EndpointEvent event, String filter)

event The event containing the details about the change.

filter The filter from the {@link #ENDPOINT_LISTENER_SCOPE} that matches (or for {@link EndpointEvent#MODIFIED_ENDMATCH} and {@link EndpointEvent#REMOVED} used to match) the endpoint, must not be {@code null}.

- Notification that an endpoint has changed. Details of the change is captured in the Endpoint Event provided. This could be that an endpoint was added, removed or modified.

122.12.5 public abstract EndpointListener

Deprecated white board service that represents a listener for endpoints. An Endpoint Listener represents a participant in the distributed model that is interested in Endpoint Descriptions. The Endpoint Listener is called back when matching endpoints are added or removed. Consumers interested in the modification of endpoints, when associated service properties are changed, should use an {@link EndpointEventListener} instead. This white board service can be used in many different scenarios. However, the primary use case is to allow a remote manager to be informed of Endpoint Descriptions available in the network and inform the network about available Endpoint Descriptions. Both the network bundle and the manager bundle register an Endpoint Listener service. The manager informs the network bundle about Endpoints that it creates. The network bundles then uses a protocol like SLP to announce these local endpoints to the network. If the network bundle discovers a new Endpoint through its discovery protocol, then it sends an Endpoint Description to all the Endpoint Listener services that are registered (except its own) that have specified an interest in that endpoint. Endpoint Listener services can express their *scope* with the service property {@link #ENDPOINT_LISTENER_SCOPE}. This service property is a list of filters. An Endpoint Description should only be given to a Endpoint Listener when there is at least one filter that matches the Endpoint Description properties. This filter model is quite flexible. For example, a discovery bundle is only interested in locally originating Endpoint Descriptions. The following filter ensure that it only sees local endpoints:

```
(org.osgi.framework.uid=72dc5fd9-5f8f-4f8f-9821-9ebb433a5b72)
```

In the same vein, a manager that is only interested in remote Endpoint Descriptions can use a filter like:

```
(!(org.osgi.framework.uid=72dc5fd9-5f8f-4f8f-9821-9ebb433a5b72))
```

Where in both cases, the given UUID is the UUID of the local framework that can be found in the Framework properties. The Endpoint Listener's scope maps very well to the service hooks. A manager can just register all filters found from the Listener Hook as its scope. This will automatically provide it with all known endpoints that match the given scope, without having to inspect the filter string. In general, when an Endpoint Description is discovered, it should be dispatched to all registered Endpoint Listener services. If a new Endpoint Listener is registered, it should be informed about all currently known Endpoints that match its scope. If a getter of the Endpoint Listener service is unregistered, then all its registered Endpoint Description objects must be removed. The Endpoint Listener models a *best effort* approach. Participating bundles should do their utmost to keep the listeners up to date, but implementers should realize that many endpoints come through unreliable discovery processes.

Deprecated @deprecated As of 1.1. Replaced by EndpointEventListener.

Concurrency Thread-safe

122.12.5.1 public static final String ENDPOINT_LISTENER_SCOPE = "endpoint.listener.scope"

Specifies the interest of this listener with filters. This listener is only interested in Endpoint Descriptions where its properties match the given filter. The type of this property must be {@code String+}.

122.12.5.2 public abstract void endpointAdded(EndpointDescription endpoint, String matchedFilter)

endpoint The Endpoint Description to be published

matchedFilter The filter from the {@link #ENDPOINT_LISTENER_SCOPE} that matched the endpoint, must not be {@code null}.

- Register an endpoint with this listener. If the endpoint matches one of the filters registered with the {@link #ENDPOINT_LISTENER_SCOPE} service property then this filter should be given as the {@code matchedFilter} parameter. When this service is first registered or it is modified, it should receive all known endpoints matching the filter.

122.12.5.3 public abstract void endpointRemoved(EndpointDescription endpoint, String matchedFilter)

endpoint The Endpoint Description that is no longer valid.

matchedFilter The filter from the {@link #ENDPOINT_LISTENER_SCOPE} that matched the endpoint, must not be {@code null}.

- Remove the registration of an endpoint. If an endpoint that was registered with the {@link #endpointAdded(EndpointDescription, String)} method is no longer available then this method should be called. This will remove the endpoint from the listener. It is not necessary to remove endpoints when the service is unregistered or modified in such a way that not all endpoints match the interest filter anymore.

122.12.6 public final class EndpointPermission extends Permission

A bundle's authority to export, import or read an Endpoint.

- The {@code export} action allows a bundle to export a service as an Endpoint.
- The {@code import} action allows a bundle to import a service from an Endpoint.
- The {@code read} action allows a bundle to read references to an Endpoint.

Permission to read an Endpoint is required in order to detect events regarding an Endpoint. Untrusted bundles should not be able to detect the presence of certain Endpoints unless they have the appropriate {@code EndpointPermission} to read the specific service.

Concurrency Thread-safe

122.12.6.1 public static final String EXPORT = "export"

The action string {@code export}. The {@code export} action implies the {@code read} action.

122.12.6.2 public static final String IMPORT = "import"

The action string {@code import}. The {@code import} action implies the {@code read} action.

122.12.6.3 public static final String READ = "read"

The action string {@code read}.

122.12.6.4 public EndpointPermission(String filterString, String actions)

filterString The filter string or "*" to match all endpoints.

actions The actions {@code read}, {@code import}, or {@code export}.

- Create a new EndpointPermission with the specified filter.

The filter will be evaluated against the endpoint properties of a requested EndpointPermission.

There are three possible actions: {@code read}, {@code import} and {@code export}. The {@code read} action allows the owner of this permission to see the presence of distributed services. The

{@code import} action allows the owner of this permission to import an endpoint. The {@code export} action allows the owner of this permission to export a service.

Throws `IllegalArgumentException`– If the filter has an invalid syntax or the actions are not valid.

122.12.6.5 **public EndpointPermission(EndpointDescription endpoint, String localFrameworkUUID, String actions)**

endpoint The requested endpoint.

localFrameworkUUID The UUID of the local framework. This is used to support matching the {@link RemoteConstants#ENDPOINT_FRAMEWORK_UUID endpoint.framework.uuid} endpoint property to the {@code <<LOCAL>>} value in the filter expression.

actions The actions {@code read}, {@code import}, or {@code export}.

- Creates a new requested {@code EndpointPermission} object to be used by code that must perform {@code checkPermission}. {@code EndpointPermission} objects created with this constructor cannot be added to an {@code EndpointPermission} permission collection.

Throws `IllegalArgumentException`– If the endpoint is {@code null} or the actions are not valid.

122.12.6.6 **public boolean equals(Object obj)**

obj The object to test for equality.

- Determines the equality of two `EndpointPermission` objects. Checks that specified object has the same name, actions and endpoint as this {@code EndpointPermission}.

Returns `true` If *obj* is a {@code EndpointPermission}, and has the same name, actions and endpoint as this {@code EndpointPermission} object; {@code false} otherwise.

122.12.6.7 **public String getActions()**

- Returns the canonical string representation of the actions. Always returns present actions in the following canonical order: {@code read}, {@code import}, {@code export}.

Returns The canonical string representation of the actions.

122.12.6.8 **public int hashCode()**

- Returns the hash code value for this object.

Returns Hash code value for this object.

122.12.6.9 **public boolean implies(Permission p)**

p The target permission to check.

- Determines if a {@code EndpointPermission} object "implies" the specified permission.

Returns {@code true} if the specified permission is implied by this object; {@code false} otherwise.

122.12.6.10 **public PermissionCollection newPermissionCollection()**

- Returns a new {@code PermissionCollection} object for storing {@code EndpointPermission} objects.

Returns A new {@code PermissionCollection} object suitable for storing {@code EndpointPermission} objects.

122.12.7 **public abstract ExportReference**

An Export Reference associates a service with a local endpoint. The Export Reference can be used to reference an exported service. When the service is no longer exported, all methods must return {@code null}.

Concurrency Thread-safe

Provider Type Consumers of this API must not implement this type

122.12.7.1 **public abstract EndpointDescription getExportedEndpoint()**

- Return the Endpoint Description for the local endpoint.

Returns The Endpoint Description for the local endpoint. Must be {@code null} when the service is no longer exported.

122.12.7.2 **public abstract ServiceReference<?> getExportedService()**

- Return the service being exported.

Returns The service being exported. Must be {@code null} when the service is no longer exported.

122.12.8 **public abstract ExportRegistration**

An Export Registration associates a service to a local endpoint. The Export Registration can be used to delete the endpoint associated with an this registration. It is created with the {@link RemoteServiceAdmin#exportService(ServiceReference, Map)} method. When this Export Registration has been closed, all methods must return {@code null}.

Concurrency Thread-safe

Provider Type Consumers of this API must not implement this type

122.12.8.1 **public abstract void close()**

- Delete the local endpoint and disconnect any remote distribution providers. After this method returns, all methods must return {@code null}. This method has no effect when this registration has already been closed or is being closed.

122.12.8.2 **public abstract Throwable getException()**

- Return the exception for any error during the export process. If the Remote Service Admin for some reasons is unable to properly initialize this registration, then it must return an exception from this method. If no error occurred, this method must return {@code null}. The error must be set before this Export Registration is returned. Asynchronously occurring errors must be reported to the log.

Returns The exception that occurred during the initialization of this registration or {@code null} if no exception occurred.

122.12.8.3 **public abstract ExportReference getExportReference()**

- Return the Export Reference for the exported service.

Returns The Export Reference for this registration, or null if this Import Registration is closed.

Throws IllegalStateException— When this registration was not properly initialized. See {@link #getException()}.

122.12.8.4 **public abstract EndpointDescription update(Map<String, ?> properties)**

properties properties to be merged with the current service properties for the {@link ServiceReference} represented by this {@link ExportRegistration}. If null is passed then the original properties passed to {@link RemoteServiceAdmin#exportService(ServiceReference, Map)} will be used.

- Update the endpoint represented by this {@link ExportRegistration} and return an updated {@link EndpointDescription}. If this method returns an updated {@link EndpointDescription}, then the object returned via {@link #getExportReference()} must also have been updated to return this new object. If this method does not return an updated {@link EndpointDescription} then the object returned via {@link #getExportReference()} should remain unchanged. When creating the updated {@link EndpointDescription} the {@link ServiceReference} originally passed to {@link RemoteServiceAdmin#exportService(ServiceReference, Map)} must be queried to pick up any changes to its service properties. If this argument is null then the original properties passed when creating this Ex-

portRegistration should be used when constructing the updated {@link EndpointDescription}. Otherwise the new properties should be used, and replace the original properties for subsequent calls to the update method.

Returns The updated {@link EndpointDescription} for this registration or null if there was a failure updating the endpoint. If a failure occurs then it can be accessed using {@link #getException()}.

Throws IllegalStateException – If this registration is closed, or when this registration was not properly initialized. See {@link #getException()}.

Since @since 1.1

122.12.9 public abstract ImportReference

An Import Reference associates an active proxy service to a remote endpoint. The Import Reference can be used to reference an imported service. When the service is no longer imported, all methods must return {@code null}.

Concurrency Thread-safe

Provider Type Consumers of this API must not implement this type

122.12.9.1 public abstract EndpointDescription getImportedEndpoint()

- Return the Endpoint Description for the remote endpoint.

Returns The Endpoint Description for the remote endpoint. Must be {@code null} when the service is no longer imported.

122.12.9.2 public abstract ServiceReference<?> getImportedService()

- Return the Service Reference for the proxy for the endpoint.

Returns The Service Reference to the proxy for the endpoint. Must be {@code null} when the service is no longer imported.

122.12.10 public abstract ImportRegistration

An Import Registration associates an active proxy service to a remote endpoint. The Import Registration can be used to delete the proxy associated with an endpoint. It is created with the {@link RemoteServiceAdmin#importService(EndpointDescription)} method. When this Import Registration has been closed, all methods must return {@code null}.

Concurrency Thread-safe

Provider Type Consumers of this API must not implement this type

122.12.10.1 public abstract void close()

- Close this Import Registration. This must close the connection to the endpoint and unregister the proxy. After this method returns, all other methods must return {@code null}. This method has no effect when this registration has already been closed or is being closed.

122.12.10.2 public abstract Throwable getException()

- Return the exception for any error during the import process. If the Remote Service Admin for some reasons is unable to properly initialize this registration, then it must return an exception from this method. If no error occurred, this method must return {@code null}. The error must be set before this Import Registration is returned. Asynchronously occurring errors must be reported to the log.

Returns The exception that occurred during the initialization of this registration or {@code null} if no exception occurred.

122.12.10.3 public abstract ImportReference getImportReference()

- Return the Import Reference for the imported service.

Returns The Import Reference for this registration, or null if this Import Registration is closed.

Throws `IllegalStateException` – When this registration was not properly initialized. See {@link #getException()}.

122.12.10.4 **public abstract boolean update(EndpointDescription endpoint)**

endpoint The updated endpoint

- Update the local service represented by this {@link ImportRegistration}. After this method returns the {@link EndpointDescription} returned via {@link #getImportReference()} must have been updated.

Returns true if the endpoint was successfully updated, false otherwise. If the update fails then the failure can be retrieved from {@link #getException()}.

Throws `IllegalStateException` – When this registration is closed, or if it was not properly initialized. See {@link #getException()}.

`IllegalArgumentException` – When the supplied {@link EndpointDescription} does not represent the same endpoint as this {@link ImportRegistration}.

Since @since 1.1

122.12.11 **public class RemoteConstants**

Provide the definition of the constants used in the Remote Service Admin specification.

Concurrency Immutable

122.12.11.1 **public static final String ENDPOINT_FRAMEWORK_UUID = "endpoint.framework.uuid"**

Endpoint property identifying the universally unique id of the exporting framework. Can be absent if the corresponding endpoint is not for an OSGi service.

The value of this property must be of type {@code String}.

122.12.11.2 **public static final String ENDPOINT_ID = "endpoint.id"**

Endpoint property identifying the id for this endpoint. This service property must always be set.

The value of this property must be of type {@code String}.

122.12.11.3 **public static final String ENDPOINT_PACKAGE_VERSION_ = "endpoint.package.version."**

Prefix for an endpoint property identifying the interface Java package version for an interface. For example, the property {@code endpoint.package.version.com.acme=1.3} describes the version of the package for the {@code com.acme.Foo} interface. This endpoint property for an interface package does not have to be set. If not set, the value must be assumed to be 0.

Since endpoint properties are stored in a case insensitive map, case variants of a package name are folded together.

The value of properties having this prefix must be of type {@code String}.

122.12.11.4 **public static final String ENDPOINT_SERVICE_ID = "endpoint.service.id"**

Endpoint property identifying the service id of the exported service. Can be absent or 0 if the corresponding endpoint is not for an OSGi service.

The value of this property must be of type {@code Long}.

122.12.11.5 **public static final String REMOTE_CONFIGS_SUPPORTED = "remote.configs.supported"**

Service property identifying the configuration types supported by a distribution provider. Registered by the distribution provider on one of its services to indicate the supported configuration types.

The value of this property must be of type {@code String}, {@code String[]}, or {@code Collection} of {@code String}.

See Also Remote Services Specification

122.12.11.6 **public static final String REMOTE_INTENTS_SUPPORTED = "remote.intents.supported"**

Service property identifying the intents supported by a distribution provider. Registered by the distribution provider on one of its services to indicate the vocabulary of implemented intents.

The value of this property must be of type {@code String}, {@code String[]}, or {@code Collection} of {@code String}.

See Also Remote Services Specification

122.12.11.7 **public static final String SERVICE_EXPORTED_CONFIGS = "service.exported.configs"**

Service property identifying the configuration types that should be used to export the service. Each configuration type represents the configuration parameters for an endpoint. A distribution provider should create an endpoint for each configuration type that it supports.

This property may be supplied in the {@code properties} {@code Dictionary} object passed to the {@code BundleContext.registerService} method. The value of this property must be of type {@code String}, {@code String[]}, or {@code Collection} of {@code String}.

See Also Remote Services Specification

122.12.11.8 **public static final String SERVICE_EXPORTED_INTENTS = "service.exported.intents"**

Service property identifying the intents that the distribution provider must implement to distribute the service. Intents listed in this property are reserved for intents that are critical for the code to function correctly, for example, ordering of messages. These intents should not be configurable.

This property may be supplied in the {@code properties} {@code Dictionary} object passed to the {@code BundleContext.registerService} method. The value of this property must be of type {@code String}, {@code String[]}, or {@code Collection} of {@code String}.

See Also Remote Services Specification

122.12.11.9 **public static final String SERVICE_EXPORTED_INTENTS_EXTRA = "service.exported.intents.extra"**

Service property identifying the extra intents that the distribution provider must implement to distribute the service. This property is merged with the {@code service.exported.intents} property before the distribution provider interprets the listed intents; it has therefore the same semantics but the property should be configurable so the administrator can choose the intents based on the topology. Bundles should therefore make this property configurable, for example through the Configuration Admin service.

This property may be supplied in the {@code properties} {@code Dictionary} object passed to the {@code BundleContext.registerService} method. The value of this property must be of type {@code String}, {@code String[]}, or {@code Collection} of {@code String}.

See Also Remote Services Specification

122.12.11.10 **public static final String SERVICE_EXPORTED_INTERFACES = "service.exported.interfaces"**

Service property marking the service for export. It defines the interfaces under which this service can be exported. This list must be a subset of the types under which the service was registered. The single value of an asterisk ({@code '*' \u002A}) indicates all the interface types under which the service was registered excluding the non-interface types. It is strongly recommended to only export interface types and not concrete classes due to the complexity of creating proxies for some type of concrete classes.

This property may be supplied in the {@code properties} {@code Dictionary} object passed to the {@code BundleContext.registerService} method. The value of this property must be of type {@code String}, {@code String[]}, or {@code Collection} of {@code String}.

See Also Remote Services Specification

122.12.11.11 **public static final String SERVICE_IMPORTED = "service.imported"**

Service property identifying the service as imported. This service property must be set by a distribution provider to any value when it registers the endpoint proxy as an imported service. A bundle can use this property to filter out imported services.

The value of this property may be of any type.

See Also Remote Services Specification

122.12.11.12 **public static final String SERVICE_IMPORTED_CONFIGS = "service.imported.configs"**

Service property identifying the configuration types used to import the service. Any associated properties for this configuration types must be properly mapped to the importing system. For example, a URL in these properties must point to a valid resource when used in the importing framework. If multiple configuration types are listed in this property, then they must be synonyms for exactly the same remote endpoint that is used to export this service.

The value of this property must be of type {@code String}, {@code String[]}, or {@code Collection} of {@code String}.

See Also Remote Services Specification, SERVICE_EXPORTED_CONFIGS

122.12.11.13 **public static final String SERVICE_INTENTS = "service.intents"**

Service property identifying the intents that this service implement. This property has a dual purpose:

- A bundle can use this service property to notify the distribution provider that these intents are already implemented by the exported service object.
- A distribution provider must use this property to convey the combined intents of: The exporting service, and the intents that the exporting distribution provider adds, and the intents that the importing distribution provider adds.

To export a service, a distribution provider must expand any qualified intents. Both the exporting and importing distribution providers must recognize all intents before a service can be distributed. The value of this property must be of type {@code String}, {@code String[]}, or {@code Collection} of {@code String}.

See Also Remote Services Specification

122.12.12 **public abstract RemoteServiceAdmin**

A Remote Service Admin manages the import and export of services. A Distribution Provider can expose a control interface. This interface allows a Topology Manager to control the export and import of services. The API allows a Topology Manager to export a service, to import a service, and find out about the current imports and exports.

Concurrency Thread-safe

Provider Type Consumers of this API must not implement this type

122.12.12.1 **public abstract Collection<ExportRegistration> exportService(ServiceReference<?> reference, Map<String, ?> properties)**

reference The Service Reference to export.

properties The properties to create a local Endpoint that can be implemented by this Remote Service Admin. If this is `{@code null}`, the Endpoint will be determined by the properties on the service. The properties are the same as given for an exported service. They override any properties in the specified Service Reference (case insensitive). The properties `{@code objectClass}` and `{@code service.id}`, in any case variant, are ignored. Those properties in the Service Reference cannot be overridden. This parameter can be `{@code null}`, this should be treated as an empty map.

- Export a service to a given Endpoint. The Remote Service Admin must create an Endpoint from the given description that can be used by other Distribution Providers to connect to this Remote Service Admin and use the exported service. The property keys of a Service Reference are case insensitive while the property keys of the specified `{@code properties}` map are case sensitive. A property key in the specified `{@code properties}` map must therefore override any case variant property key in the properties of the specified Service Reference.

If the caller does not have the appropriate `{@code EndpointPermission[endpoint,EXPORT]}` for an Endpoint, and the Java Runtime Environment supports permissions, then the `{@link ExportRegistration#getException() getException}` method on the corresponding returned `{@link ExportRegistration}` will return a `{@code SecurityException}`.

Returns A `{@code Collection}` of `{@link ExportRegistration}`s for the specified Service Reference and properties. Multiple Export Registrations may be returned because a single service can be exported to multiple Endpoints depending on the available configuration type properties and the intents that they support. The result is never `{@code null}` but may be empty if this Remote Service Admin does not recognize any of the configuration types, or if the Remote Service Admin cannot support the relevant intents.

Throws `IllegalArgumentException`– If any of the properties for this configuration type has a value that is not syntactically correct, or if the service properties and the overlaid properties do not contain a `{@link RemoteConstants#SERVICE_EXPORTED_INTERFACES}` entry. This means that implementations must not ignore invalid values for property names that they recognize.

122.12.12.2 **public abstract Collection<ExportReference> getExportedServices()**

- Return the currently active Export References.

If the caller does not have the appropriate `{@code EndpointPermission[endpoint,READ]}` for an Endpoint, and the Java Runtime Environment supports permissions, then returned collection will not contain a reference to the exported Endpoint.

Returns A `{@code Collection}` of `{@link ExportReference}`s that are currently active.

122.12.12.3 **public abstract Collection<ImportReference> getImportedEndpoints()**

- Return the currently active Import References.

If the caller does not have the appropriate `{@code EndpointPermission[endpoint,READ]}` for an Endpoint, and the Java Runtime Environment supports permissions, then returned collection will not contain a reference to the imported Endpoint.

Returns A `{@code Collection}` of `{@link ImportReference}`s that are currently active.

122.12.12.4 **public abstract ImportRegistration importService(EndpointDescription endpoint)**

endpoint The Endpoint Description to be used for import.

- Import a service from an Endpoint. The Remote Service Admin must use the given Endpoint to create a proxy. This method can return `{@code null}` if the service could not be imported.

Returns An Import Registration that combines the Endpoint Description and the Service Reference or `{@code null}` if the Endpoint could not be imported.

Throws `SecurityException`– If the caller does not have the appropriate `{@code EndpointPermission[endpoint,IMPORT]}` for the Endpoint, and the Java Runtime Environment supports permissions.

122.12.13 public class RemoteServiceAdminEvent

Provides the event information for a Remote Service Admin event.

Concurrency Immutable

122.12.13.1 public static final int EXPORT_ERROR = 6

A fatal exporting error occurred. The Export Registration has been closed.

122.12.13.2 public static final int EXPORT_REGISTRATION = 2

Add an export registration. The Remote Service Admin will send this event when it exports a service. When the {@link RemoteServiceAdminListener} service is registered, the Remote Service Admin must notify the listener of all existing Export Registrations.

122.12.13.3 public static final int EXPORT_UNREGISTRATION = 3

Remove an export registration. The Remote Service Admin will send this event when it removes the export of a service.

122.12.13.4 public static final int EXPORT_UPDATE = 10

Update an export registration. The Remote Service Admin will send this event when it exports a service.

Since @since 1.1

122.12.13.5 public static final int EXPORT_WARNING = 7

A problematic situation occurred, the export is still active.

122.12.13.6 public static final int IMPORT_ERROR = 5

A fatal importing error occurred. The Import Registration has been closed.

122.12.13.7 public static final int IMPORT_REGISTRATION = 1

Add an import registration. The Remote Service Admin will send this event when it imports a service. When the {@link RemoteServiceAdminListener} service is registered, the Remote Service Admin must notify the listener of all existing Import Registrations.

122.12.13.8 public static final int IMPORT_UNREGISTRATION = 4

Remove an import registration. The Remote Service Admin will send this event when it removes the import of a service.

122.12.13.9 public static final int IMPORT_UPDATE = 9

Update an import registration. The Remote Service Admin will send this event when it updates a service.

Since @since 1.1

122.12.13.10 public static final int IMPORT_WARNING = 8

A problematic situation occurred, the import is still active.

122.12.13.11 public RemoteServiceAdminEvent(int type, Bundle source, ExportReference exportReference, Throwable exception)

type The event type.

source The source bundle, must not be {@code null}.

exportReference The exportReference, can not be {@code null}.

exception Any exceptions encountered, can be {@code null}.

- Create a Remote Service Admin Event for an export notification.

122.12.13.12 **public RemoteServiceAdminEvent(int type, Bundle source, ImportReference importReference, Throwable exception)**

type The event type.

source The source bundle, must not be {@code null}.

importReference The importReference, can not be {@code null}.

exception Any exceptions encountered, can be {@code null}.

- Create a Remote Service Admin Event for an import notification.

122.12.13.13 **public Throwable getException()**

- Return the exception for this event.

Returns The exception or {@code null}.

122.12.13.14 **public ExportReference getExportReference()**

- Return the Export Reference for this event.

Returns The Export Reference or {@code null}.

122.12.13.15 **public ImportReference getImportReference()**

- Return the Import Reference for this event.

Returns The Import Reference or {@code null}.

122.12.13.16 **public Bundle getSource()**

- Return the bundle source of this event.

Returns The bundle source of this event.

122.12.13.17 **public int getType()**

- Return the type of this event.

Returns The type of this event.

122.12.14 **public abstract RemoteServiceAdminListener**

A {@link RemoteServiceAdminEvent} listener is notified synchronously of any export or import registrations and unregistrations.

If the Java Runtime Environment supports permissions, then filtering is done. {@code RemoteServiceAdminEvent} objects are only delivered to the listener if the bundle which defines the listener object's class has the appropriate {@code EndpointPermission[endpoint,READ]} for the endpoint referenced by the event.

See Also RemoteServiceAdminEvent

Concurrency Thread-safe

122.12.14.1 **public abstract void remoteAdminEvent(RemoteServiceAdminEvent event)**

event The {@link RemoteServiceAdminEvent} object.

- Receive notification of any export or import registrations and unregistrations as well as errors and warnings.

122.13 org.osgi.service.remoteserviceadmin.namespace

Remote Service Admin Namespaces Version 1.0.

Bundles should not need to import this package at runtime since all the types in this package just contain constants for capability and requirement namespaces specified by the OSGi Working Group.

122.13.1 Summary

- `DiscoveryNamespace` - Remote Services Discovery Provider Capability and Requirement Namespace.
- `DistributionNamespace` - Remote Services Distribution Provider Capability and Requirement Namespace.
- `TopologyNamespace` - Remote Services Topology Manager Capability and Requirement Namespace.

122.13.2 **public final class** `DiscoveryNamespace` **extends** `Namespace`

Remote Services Discovery Provider Capability and Requirement Namespace.

This class defines the names for the attributes and directives for this namespace.

Concurrency Immutable

122.13.2.1 **public static final** `String` `CAPABILITY_PROTOCOLS_ATTRIBUTE` = "protocols"

The capability attribute used to specify the discovery protocols supported by this discovery provider. The value of this attribute must be of type {@code String} or {@code List<String>}.

122.13.2.2 **public static final** `String` `DISCOVERY_NAMESPACE` = "org.osgi.service.remoteserviceadmin.discovery"

Namespace name for Remote Services discovery provider capabilities and requirements.

122.13.3 **public final class** `DistributionNamespace` **extends** `Namespace`

Remote Services Distribution Provider Capability and Requirement Namespace.

This class defines the names for the attributes and directives for this namespace.

Concurrency Immutable

122.13.3.1 **public static final** `String` `CAPABILITY_CONFIGS_ATTRIBUTE` = "configs"

The capability attribute used to specify the config types supported by this distribution provider. The value of this attribute must be of type {@code String} or {@code List<String>}.

122.13.3.2 **public static final** `String` `DISTRIBUTION_NAMESPACE` = "org.osgi.service.remoteserviceadmin.distribution"

Namespace name for Remote Services distribution provider capabilities and requirements.

122.13.4 **public final class** `TopologyNamespace` **extends** `Namespace`

Remote Services Topology Manager Capability and Requirement Namespace.

This class defines the names for the attributes and directives for this namespace.

Concurrency Immutable

122.13.4.1**public static final String CAPABILITY_POLICY_ATTRIBUTE = "policy"**

The capability attribute used to specify the policy or policies supported by this topology manager. The value of this attribute must be of type {@code String} or {@code List<String>}. Policy names are typically implementation specific, however the Remote Services Specification defines the *promiscuous* and *fail-over* policies for common use cases.

122.13.4.2**public static final String FAIL_OVER_POLICY = "fail-over"**

The attribute value for Topology managers with a fail-over policy

See Also TopologyNamespace.CAPABILITY_POLICY_ATTRIBUTE

122.13.4.3**public static final String PROMISCUOUS_POLICY = "promiscuous"**

The attribute value for Topology managers with a promiscuous policy

See Also TopologyNamespace.CAPABILITY_POLICY_ATTRIBUTE

122.13.4.4**public static final String TOPOLOGY_NAMESPACE = "osgi.remoteserviceadmin.topology"**

Namespace name for Remote Services topology manager capabilities and requirements.

122.14 References

- [1] *OSGi Service Property Namespace*
<https://docs.osgi.org/reference/service-property-namespace.html>
- [2] *UUIDs*
https://en.wikipedia.org/wiki/Universally_Unique_Identifier
- [3] *Service Location Protocol (SLP)*
https://en.wikipedia.org/wiki/Service_Location_Protocol
- [4] *JGroups*
<http://www.jgroups.org/>
- [5] *UDDI*
https://en.wikipedia.org/wiki/Universal_Description_Discovery_and_Integration
- [6] *Service Component Architecture (SCA)*
<https://www.osoa.org/>

End Of Document

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT